NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

MARCOS VINICIUS BONFIM ROMÃO

BSc in Electrical and Computer Engineering

5G NETWORKS: DEPLOYMENT OF AN EXPERIMENTAL 5G CORE AND RADIO ACCESS NETWORK

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING

NOVA University Lisbon

Draft: *August 29, 2026*

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

5G NETWORKS: DEPLOYMENT OF AN EXPERIMENTAL 5G CORE AND RADIO ACCESS NETWORK

MARCOS VINICIUS BONFIM ROMÃO

BSc in Electrical and Computer Engineering

Supervisor: Rodolfo Oliveira

Full Professor, NOVA University Lisbon

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING

NOVA University Lisbon

Draft: *August 29, 2026*

Contents

List of Figures

List of Tables

Listings

List of Listings

Acronyms

3GPP 3rd Generation Partnership Project *(pp. 9, 14, 20, 21, 29, 30)*

AF Application Function *(p. 18)*

AMF Access and Mobility Management Function *(pp. 7, 16, 19, 20)*

API Application Programming Interface *(pp. 16, 18, 19)*

AUSF Authentication Server Function *(p. 18)*

AWS Amazon Web Services *(p. 7)*

Azure Microsoft Azure *(p. 7)*

CD/CI Continuous Development/Continuous Integration *(p. 24)*

COTS Commercial Off-The-Shelf *(pp. 3, 15, 23, 24, 26, 57)*

CU Centralized Unit *(pp. 11–13, 21, 22, 30)*

CUPS Control and User Plane Separation *(pp. i, 6, 16)*

DN Data Network *(p. 19)*

DU Distributed Unit *(pp. 11–13, 21, 22)*

eMBB enhanced Mobile Broadband *(p. 10)*

eNodeB Evolved Node B *(pp. 11, 57, 71, 72)*

EPC Evolved Packet Core *(pp. 2, 5, 9, 16–18, 20, 56, 57, 59, 62, 71, 72)*

F1-U F1 User Plane Interface *(p. 14)*

F1AP F1 Application Protocol *(p. 14)*

FCT NOVA School of Science and Technology *(p. 3)*

FPGA Field-Programmable Gate Array *(p. 29)*

GCP Google Cloud Platform *(p. 7)*

gNodeB Next Generation Node B *(pp. iv, 2, 11, 13, 14, 21–25)*

GPSDO GPS Disciplined Oscillator *(pp. 24, 29)*

GSM Global System for Mobile Communications *(p. 24)*

GTP-C GPRS Tunneling Protocol - Control Plane *(p. 19)*

GTP-U GPRS Tunneling Protocol - User Plane *(p. 19)*

HSS Home Subscriber Server *(p. 17)*

IoT Internet of Things *(pp. 1, 7)*

LMF Location Management Function *(p. 18)*

LNA Low Noise Amplifier *(pp. iv, 33, 34, 58)*

LPWA Low-Power Wide-Area *(p. 7)*

LTE Long-Term Evolution *(pp. 14, 22, 24)*

MAC Medium Access Control *(p. 11)*

MIMO Multiple Input Multiple Output *(p. 14)*

MME Mobility Management Entity *(p. 16)*

mMTC massive Machine Type Communications *(p. 10)*

mmWave Millimeter Wave *(p. 15)*

N3IWF Non-3GPP Interworking Function *(p. 18)*

NEF Network Exposure Function *(pp. 17, 18)*

NG-C NG Control Plane Interface *(p. 19)*

NG-RAN Next Generation Radio Access Network *(pp. i, 10, 11)*

NGAP NG Application Protocol *(p. 19)*

NR New Radio *(pp. vii, 14, 30, 34)*

NRF Network Repository Function *(pp. 6, 17)*

NSA Non-Standalone *(pp. i, iv, 8, 9, 20, 22, 24, 31, 32)*

NSSF Network Slice Selection Function *(p. 18)*

NWDAF Network Data Analytics Function *(p. 18)*

OAI OpenAirInterface *(pp. 21, 26)*

OSS Open-Source Software *(pp. 19, 23, 25)*

OTA Over-The-Air *(p. 24)*

PCF Policy Control Function *(pp. 17, 20)*

PCRF Policy and Charging Rules Function *(p. 17)*

PDCP Packet Data Convergence Protocol *(p. 11)*

PDU Protocol Data Unit *(pp. 17, 24)*

PHY Physical Layer *(p. 11)*

QoS Quality of Service *(pp. 11, 14, 18)*

RAN Radio Access Network *(pp. ii, vii, 1, 4, 7, 9, 10, 13, 18, 19, 21–25, 30–32, 34–36, 39, 42, 56–59, 68, 70–72)*

RF Radio Frequency *(p. 29)*

RLC Radio Link Control *(p. 11)*

RRC RAN Control Protocol *(p. 11)*

RSP Remote SIM Provisioning *(p. 73)*

RU Radio Unit *(p. 12)*

SA Standalone *(pp. i, 1–3, 8, 9, 20, 22–24)*

SBA Service-Based Architecture *(pp. i, 6, 9)*

SCEF Service Capability Exposure Function *(p. 18)*

SCS Subcarrier Spacing *(p. 47)*

SCTP Stream Control Transmission Protocol *(p. 58)*

SDR Software-Defined Radio *(pp. 2, 3, 22–25, 28–30, 57, 58, 67)*

SMF Session Management Function *(pp. 17, 19, 20)*

SMSF Short Message Service Function *(p. 18)*

UDM Unified Data Management *(pp. 17, 18)*

UDR Unified Data Repository *(pp. 17, 18)*

UE User Equipment *(pp. ii, v, vii, 7, 10, 12–14, 16–19, 22–25, 30, 34, 36–39, 41, 42, 57, 71, 72)*

UPF User Plane Function *(pp. 17–20)*

URLLC Ultra-Reliable Low-Latency Communications *(pp. 7, 9, 10)*

VM Virtual Machine *(p. 7)*

1

Introduction

This chapter introduces the work developed in this dissertation. Section 1.1 presents the motivation behind it, Section 1.1.1 states its objectives and expected contributions, and Section 1.2 outlines the structure of the remainder of the document.

1.1 Motivation

5G networks are no longer just an incremental evolution of mobile broadband; they are the technological foundation for a new generation of applications that demand infallible reliability, ultra-low latency and massive connectivity. From industrial automation [20, 10, 7], immersive extended and augmented reality [34, 21, 31] to Internet of Things (IoT) applications [11, 36], these use cases require a network architecture that can be reconfigured, scaled and tailored to their specific needs, placing enormous emphasis on programmability and softwarization across both the RAN and Core.

Yet for a university research group, or any organisation without a commercial operator's budget, deploying 5G to actually explore these use cases is out of reach. 5G infrastructure imposes a significant capital barrier: an IEEE Access study estimated that a full 5G rollout across the United Kingdom alone would cost between £30 billion and £50 billion, dwarfing the £2.5 billion mobile network operators spend annually on capital expenditure [26]. Beyond the raw cost, commercial 5G infrastructure is closed by design, so research conducted on it is constrained by reproducibility limitations for researchers without access to the underlying, proprietary implementation. These financial and accessibility barriers have historically restricted real-world 5G Standalone (SA) research to a small

circle of well-funded operators and vendors, slowing innovation and deployment diversity, and have pushed research institutions and academic investigators instead towards open-source, software-defined alternatives, such as srsRAN [27], OpenAirInterface [6] and free5GC [13], combined with commodity software-defined radio hardware [18, 33, 22]. Testbeds combining these open-source core platforms with software-defined radios on commodity servers have repeatedly demonstrated the feasibility of low-cost 5G network implementations [8], and these platforms make it possible to validate architectural choices, reproduce results and teach the internals of a technology that, in its commercial form, is deliberately opaque.

However, deploying a fully functional SA 5G network from open-source components alone is still a non-trivial undertaking, and getting it running is only half the problem: without a clear performance baseline, it is difficult to tell whether a given result reflects a genuine property of 5G, a limitation of the specific open-source stack, or simply a misconfigured radio. The existing literature tends to report on open-source 5G deployments in isolation, publishing throughput and latency figures with no directly comparable predecessor built with the same tools, on the same hardware, to put those numbers in context; this lack of publicly available, comparable results has been noted directly in the literature [30]. A research group evaluating whether an open-source 5G testbed is viable for its needs is largely left to take those figures on faith.

This dissertation addresses that gap by building not one but two open-source, Software-Defined Radio (SDR)-based cellular networks side by side: a 5G SA network built on the OCUDU gNodeB [27] and Open5GS [25], and a 4G network built on srsRAN 4G [1] and the same Open5GS core, running its Evolved Packet Core (EPC) components. Sharing the core hardware and, as far as the technologies allow, the test methodology between the two generations makes it possible to treat the 4G network as a genuine baseline for the 5G one, rather than presenting 5G performance figures with nothing to anchor them against. By relying exclusively on open-source software and commodity hardware, this work also aims to show that meaningful 5G SA experimentation does not require a commercial operator's budget, lowering the barrier to entry for future 5G research at institutions in a similar position. Among the objectives of this dissertation is also to leave, at its conclusion, a working 4G and 5G network testbed in place at the university, so that future projects can

build on and explore it directly instead of starting from scratch.

1.1.1 Objectives and Contributions

This subsection outlines the primary objectives defined for this dissertation, as well as the contributions expected from it.

Objectives:

- Review the architecture, protocols and open-source implementations available for 5G networks, and the equivalent 4G components needed to build a comparable baseline.
- Design and deploy an open-source, SDR-based 5G SA network and a 4G network, sharing the same core network hardware, using accessible Commercial Off-The-Shelf (COTS) equipment throughout.
- Define and execute a test methodology capable of empirically comparing the two generations across multiple locations, traffic conditions and directions.
- Diagnose and iteratively correct configuration issues surfaced during testing, documenting the reasoning behind each change rather than only the final configuration.
- Leave a working, documented 4G and 5G testbed in place for future research and coursework at NOVA School of Science and Technology (FCT).

Expected Contributions:

- A functional, dual-generation 4G and 5G testbed, built entirely from open-source software and COTS SDR hardware, reusable by future projects.
- An empirical, multi-round comparison of 4G and 5G performance, throughput, latency and reliability, under matched distance and traffic conditions.
- A documented account of how specific configuration issues, namely the TDD slot structure and receiver gain staging, were diagnosed and corrected across successive test rounds, intended as a practical reference for others deploying a similar open-source stack.

1.2 Report Structure

The remainder of this document is structured as follows:

- Chapter 2 introduces the fundamental concepts and architecture of 5G networks, covering both the RAN and the core, as background for the chapters that follow.
- Chapter 3 surveys existing open-source 5G implementations and related testbed deployments described in the literature.
- Chapter 4 presents the architecture originally proposed for the testbed: the hardware components, the core network design, the RAN design and the interfaces connecting them.
- Chapter 5 describes how the testbed was actually built, including deviations from the proposed architecture, and details the methodology used to evaluate it across three successive test rounds.
- Chapter 6 presents the results of the performance evaluation, comparing the 4G baseline against the 5G network as it evolved across test rounds.
- Chapter **??** concludes the dissertation, summarising the main findings, their limitations, and the work left open for the future.

2

Fundamentals of a 5G Network

2.1 Introduction to the 5G Architecture

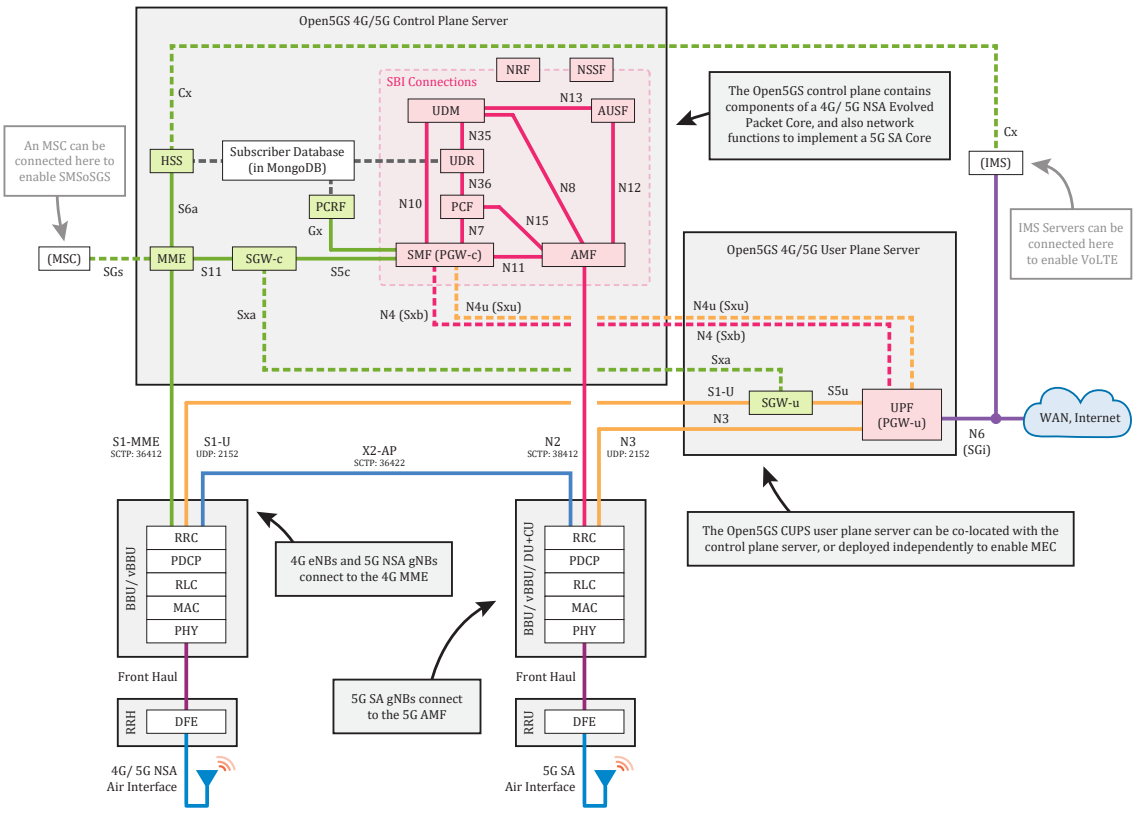


Figure 2.1: 5G Network architecture diagram from the Open5GS project [25]. Used under the AGPL-3.0 license.

The 5G architecture represents a significant evolution from previous generations of mobile networks. Compared to 4G's EPC, the 5G architecture is designed to be more flexible, scalable and efficient, enabling a wide range of novel use cases and services. Key features

of the 5G architecture include:

2.1.1 Service-Based Architecture

Unlike the monolithic architecture of 4G and previous generations, 5G adopts a service-based approach, where network functions are designed to be modular services that can be independently configured and implemented. These network functions expose their capabilities and communicate via HTTP/2, using a standardized REST API, allowing for interoperability between services.

In this paradigm, the functions take the roles of either Service Providers or Service Consumers, depending on whether they offer or utilize specific services. The Network Repository Function (NRF), which will be developed upon further, plays a crucial role in this architecture by maintaining a registry of available network functions and their services, enabling dynamic discovery and interaction among them.

2.1.2 Control and User Plane Separation

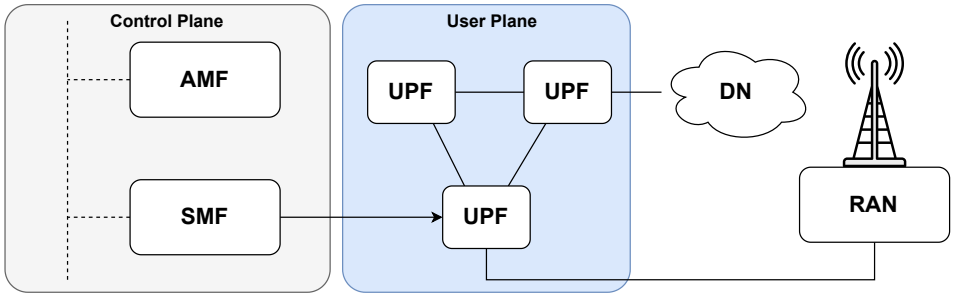


Figure 2.2: Control and User Plane Separation.

In 5G systems, the separation of the control plane and user plane is one of the fundamental design principles that sets them apart from previous generations, defining them as independent functional entities that can be deployed, scaled and managed separately, as shown on figure 2.2.

This means that network operators can dimension their networks to adapt to real network traffic patterns, deploying control plane functions in centralized locations for efficient signaling, while user plane functions can be distributed closer to the network

edge to minimize service latency and jittering, improving throughput and overall user experience.

2.1.3 Network Slicing

The concept of network slicing isn't new, but 5G develops upon it, driven by the rising popularity of odd use cases, such as Low-Power Wide-Area (LPWA) networks, IoT devices and Ultra-Reliable Low-Latency Communications (URLLC) applications.

Network slicing allows operators to create and deploy multiple virtual networks on top of a shared physical infrastructure, each optimized for specific use cases and services. 5G network slicing also allows operators to dynamically allocate resources based on the subscription and applications running on the UE.

In contrast to 4G EPS slicing, a 5GS network slice consists of both RAN and Core components and allows for multiple concurrent slices to be active on a single UE, with the only drawback being that the RAN and Access and Mobility Management Function (AMF) must be shared across those network slices.

2.1.4 Virtualization

Traditionally, components of a mobile network were tied to hardware appliances that only ran a specific function. This meant that software had to be pre-installed and configured for each network function, leading to inefficiencies in resource utilization and scalability.

In 5G, network functions can be virtualized, running in Virtual Machines (VMs) or containers, allowing them to share physical hardware with other functions. This approach provides greater flexibility and resource optimization, since functions are no longer tied to the hardware and can be dynamically allocated and decommissioned given current network demands.

2.1.5 Cloud-Native Functions

Building upon virtualization, 5G network functions are designed to be cloud-native, meaning they are developed from scratch and optimized to be run in cloud environments, such as public clouds (e.g. Amazon Web Services, Google Cloud Platform, Microsoft Azure, etc.) or private operator clouds.

This means that network operators can run part of their 5G network in cloud environments, taking advantage of their resilience and scalability, as well as the ability to deploy network functions closer to end-users, given their global reach, reducing network latency.

Being cloud-native, network operators can also take advantage of orchestration and automation tools, such as Kubernetes and OpenStack [22], to manage the lifecycle of network functions, deployment, scaling and healing, further enhancing the efficiency of their 5G networks.

2.2 Standalone vs Non-Standalone 5G Deployments

The adoption of 5G technology has been a gradual process for many network operators, as the advent of 5G networks, while innovative, also meant a significant revamp in existing infrastructure, which brought along considerable costs, logistical challenges and unavoidable service disruptions. Before evolving into fully independent 5G networks, many operators opted for integrating 5G radio capabilities into their existing 4G infrastructure. This hybrid approach is aptly named Non-Standalone, whereas completely independent 5G networks are known as Standalone deployments. In this section, we explore the differences between these two deployment modes.

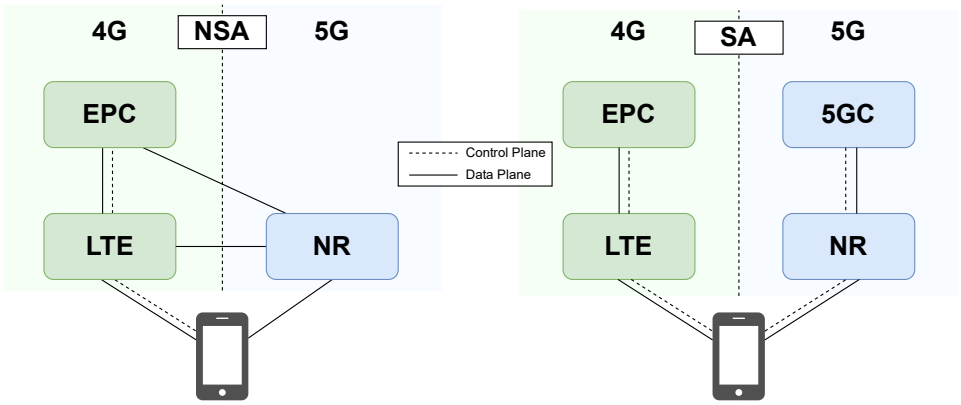


Figure 2.3: 5G NSA vs SA Deployments

2.2.1 Non-Standalone (NSA) 5G Deployments

Non-Standalone deployments were introduced as a way to accelerate the rollout of 5G services while keeping costs minimal, by leveraging existing 4G infrastructure while

implementing 5G RAN components. Standardized by the 3rd Generation Partnership Project (3GPP) under Deployment Option 3, in this architecture, the 4G EPC continues to handle control plane functions, such as mobility management and session control, but user plane data is transmitted over the 5G RAN, allowing users to benefit from the enhanced data throughput of 5G technology.

However, this approach is fundamentally constrained by its reliance on the legacy 4G core. Consequently, it cannot leverage the advanced, cloud-native capabilities of the 5G system described in Section 2.1, such as network slicing or the Service-Based Architecture (SBA). Furthermore, while CapEx is initially kept low, NSA deployments significantly increase network complexity. Operators are forced to manage, synchronize, and maintain both 4G and 5G nodes concurrently, leading to elevated operational expenditures (OpEx). Ultimately, NSA served as a necessary transitional phase, allowing operators to monetize early 5G radio investments while preparing for the complex migration to SA architectures.

2.2.2 Standalone (SA) 5G Deployments

Standalone deployments represent the true, end-to-end realization of the 5G system as originally defined by 3GPP Release 15 [2] and expanded in subsequent releases. Formally known as 3GPP Deployment Option 2, this architecture completely decouples from legacy LTE networks. Both the Radio Access Network and the Core are built entirely on 5G protocols, utilizing the full suite of cloud-native network functions. In this model, all control plane and user plane traffic is natively routed through the 5G Core (5GC), fully unlocking the benefits of the Service-Based Architecture.

Although SA deployments require a substantial upfront investment, they offer critical technical advantages over NSA architectures. Operating a native 5G environment is the only way to facilitate strict URLLC and dynamically instantiate end-to-end network slices for enterprise use cases. By eliminating the reliance on legacy 4G nodes, SA networks reduce long-term signaling overhead and are inherently more future-proof, allowing operators to seamlessly integrate emerging 3GPP standards and dynamically scale resources to meet evolving market demands.

2.3 5G RAN

The 5G Radio Access Network constitutes the interface between UEs and the 5G Core network, providing the radio connectivity and resource management functions essential to mobile communication. It encompasses the physical, MAC and higher layers of the protocol stack, enabling data transmission, mobility management and strict service differentiation. Beyond providing radio connectivity, the 5G RAN plays a central role in facilitating the diverse service categories defined for 5G networks, such as enhanced Mobile Broadband (eMBB), URLLC and massive Machine Type Communications (mMTC). To meet these requirements, the 5G RAN has evolved into a highly flexible, programmable architecture, capable of operating across a wide range of frequency bands, while dynamically adapting its numerology, scheduling and resource allocation strategies.

The NG-RAN, compared to its predecessors, has undergone extensive softwarization and disaggregation of the RAN. This archirectural shift replaces fixed hardware entities with modular components and virtualized functions that can be deployed in cloud-native environments. As a result, in this generation, the RAN acts as a dynamic platform that interacts closely with the 5G Core network and its functions to deliver optimized connectivity and services.

This chapter provides a detailed overview of the 5G RAN architecture and features, highlighting its key components and functionalities.

2.3.1 NG-RAN Architecture

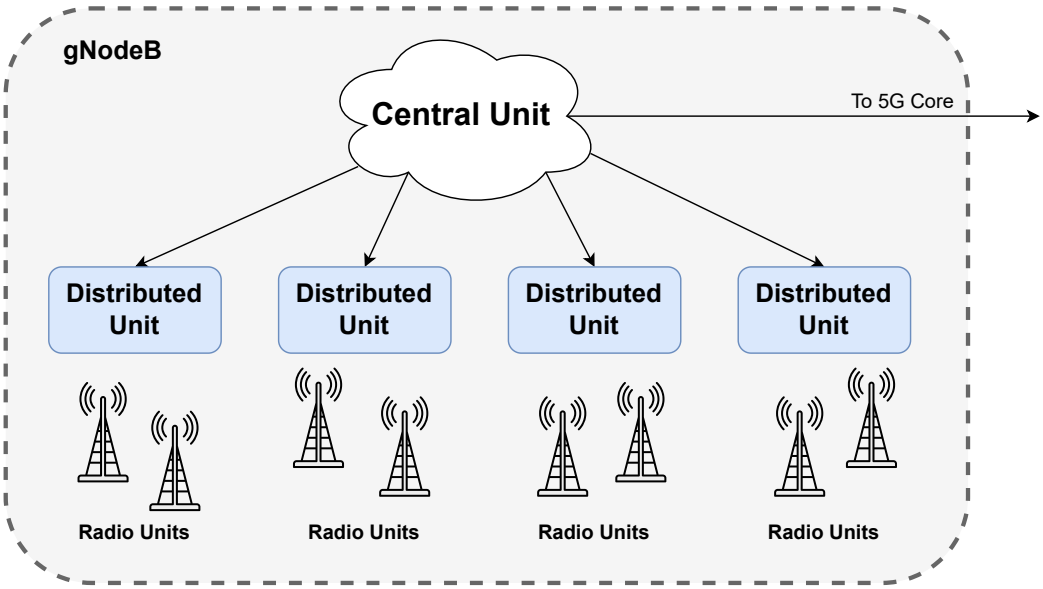


Figure 2.4: gNodeB Architecture [32]

The architecture of the NG-RAN has been fundamentally redesigned to maximize deployment flexibility by disaggregating the traditional base station functions into modular logical nodes. At its core, the NG-RAN is built around the Next Generation Node B (gNodeB) (Figure 2.4), the 5G equivalent of 4G'S eNodeB, responsible for radio transmission, user-plane data forwarding and control-plane signaling with the 5G Core network.

The gNodeB is further divided into three main primary units:

Centralized Unit

This unit is responsible for higher-layer protocol processing, specifically the RAN Control Protocol (RRC) and Packet Data Convergence Protocol (PDCP) layers, handling mobility, security and Quality of Service (QoS) flows, and is typically deployed in centralized locations and/or cloud environments. The Centralized Unit (CU) can manage multiple Distributed Units (DUs), coordinating resource usage across a wide area.

Distributed Unit

The DU handles lower-layer protocol processing, including Radio Link Control (RLC), Medium Access Control (MAC) and Physical Layer (PHY) functions. Multiple DUs

can be connected to the same CU. They are strategically deployed closer to the cell sites to minimize latency and optimize radio resource management.

Radio Unit

The Radio Units (RUs) are responsible for the physical radio frequency functions, including signal transmission, reception, amplification, and antenna management. These are composed of the actual antennas which interact with the UEs, along with the necessary back-end, and they are typically located at the cell site. They are connected to the DU through front-haul links.

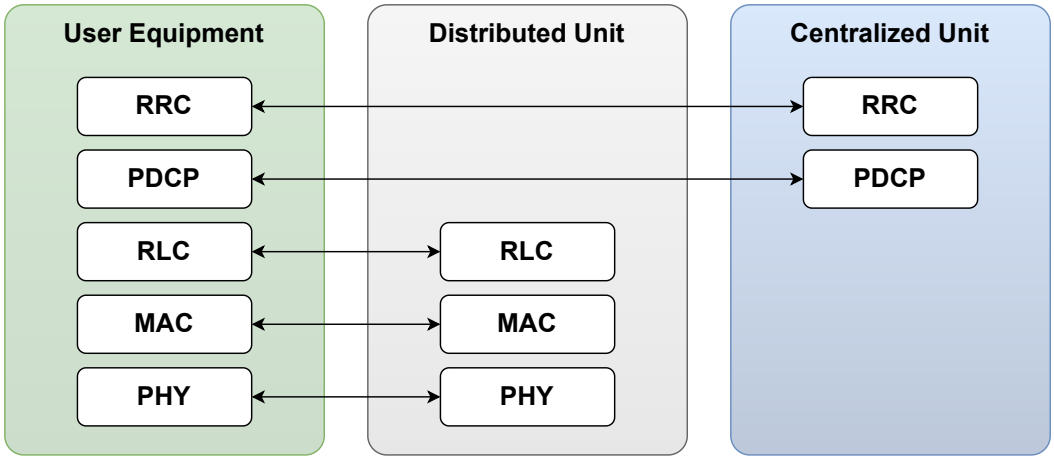


Figure 2.5: 5G RAN Protocol Stack [32]

2.3.2 RAN Interfaces

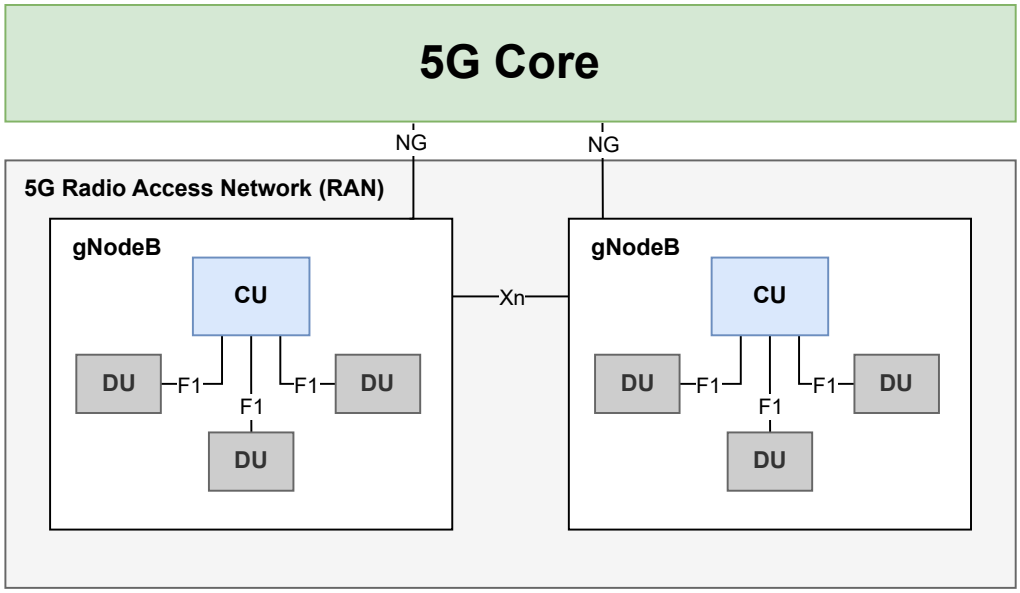


Figure 2.6: 5G RAN [32]

As depicted in Figure 2.6, the 5G RAN utilizes standardized interfaces to facilitate communication between its components and the 5G Core network. The primary interfaces include:

NG Interface

The NG interface connects the gNodeB to the 5G Core. It is structurally divided into two sub-interfaces: NG-C for control plane signaling with the AMF, and NG-U for user plane data transfer with the UPF, facilitating both control plane and user plane communications.

Xn Interface

The Xn interface connects a gNodeBs between each other, enabling inter-cell coordination and handover management. This interface also supports dual connectivity scenarios, where a UE can be simultaneously connected to multiple gNodeBs for improved performance and reliability.

F1 Interface

The F1 interface bridges the DUs to the CU within the disaggregated gNodeB. It is

also divided into two sub-interfaces: F1 User Plane Interface (F1-U) for user plane data and F1 Application Protocol (F1AP) for control plane signaling.

2.3.3 RAN Features

5G NR develops upon pre-existing technologies, introduced in previous generations, while introducing new features to meet the ever-demanding requirements of modern mobile communications. One feature that differs NR from its predecessors is its Multiple Input Multiple Output (MIMO) technology, introducing Massive MIMO.

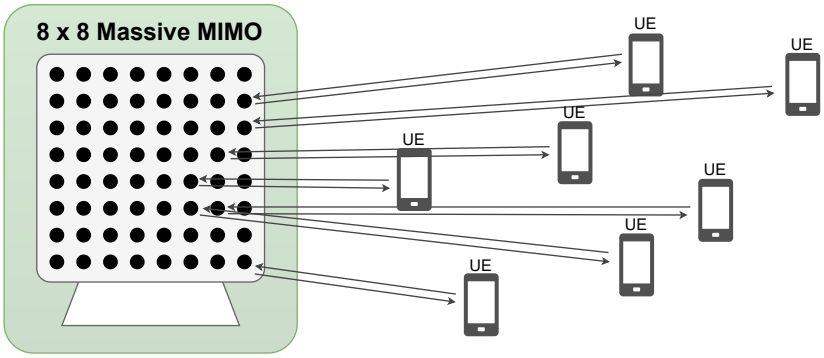


Figure 2.7: Massive MIMO antennas

In 4G LTE, MIMO capabilities were already employed, dividing the available antennas between different spatial cells to increase spectral efficiency and throughput. However, 5G NR takes this concept further by significantly increasing the number of antennas at the base station, often with upwards of 100+ antennas per gNodeB (Figure 2.7), allowing for User Equipments to be served simultaneously by a single node using the same time-frequency resources.

Each antenna is capable of beamforming, which focuses the radio signal directly towards the UE, rather than broadcasting it omnidirectionally. By dynamically adjusting the phase and amplitude of the signal at each antenna, the system can create highly directional beams that target specific users or cells. This not only improves signal strength and throughput, but also reduces wasted energy and interference with other devices, enhancing the QoS.

Driven by the increasing demand for higher bandwidths, 3GPP Release 17 [3] extended the operation spectrum up to 71 GHz, subdividing the frequency ranges into two categories,

as shown in Figure 2.8.

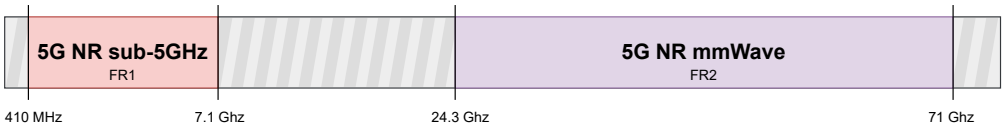


Figure 2.8: 5G Frequency Bands

- **Frequency Range 1 (FR1)**: Known as the Sub-6GHz range, it encompasses frequencies from 410 MHz to 7.125 GHz.

- **Frequency Range 2 (FR2)**: Known as mmWave, it covers frequencies from 24.25 GHz to 71 GHz. This range greatly increases available bandwidth and, therefore, throughput, but suffers from significant path loss. It is important to note that many COTS devices are not compatible with the frequency range.

2.4 5G Core

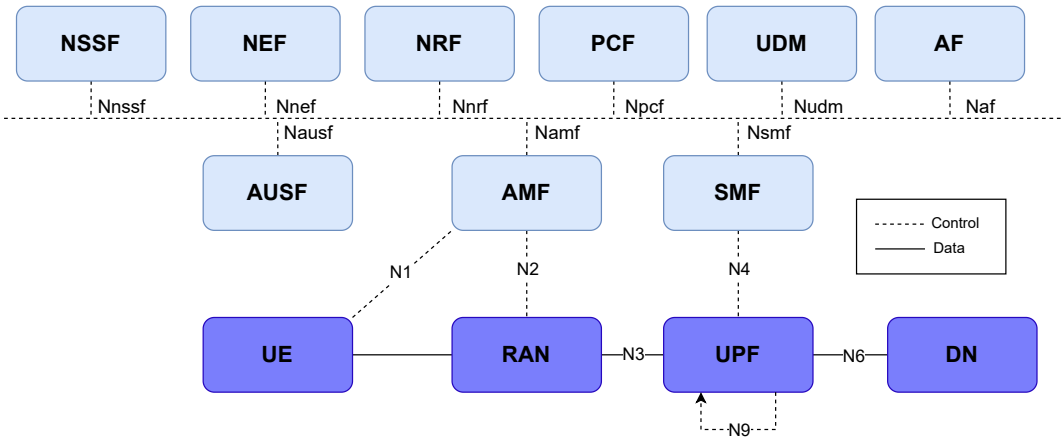


Figure 2.9: 5G Core Network Architecture [32]

As mentioned before [2.1.1], a key difference between the 5G Core network and the EPC used in 4G networks is the adoption of a service-based architecture. Network Functions are main components that form the 5G Core, and are designed to be modular and communicate with each other using standardized Application Programming Interfaces (APIs), allowing for greater flexibility, scalability, and the ability to introduce new services easily.

2.4.1 Core Network Functions

As discussed in Section 2.1.2, Control and User Plane Separation (CUPS) defines the separation of control plane and user plane functions. As defined by Rommer et al. [29], the main control plane functions are described below:

2.4.1.1 Control Plane

Access and Mobility Management Function

The AMF is a key control plane function in the 5G Core network, counterpart of the EPC's Mobility Management Entity (MME). It is responsible for handling connection and mobility management tasks for the UE. It manages registration, authentication, reachability, and mobility procedures, ensuring seamless connectivity as users move across different network areas. The AMF also interfaces with other network functions to facilitate session management and policy enforcement.

Session Management Function

The Session Management Function (SMF) is the function responsible for establishing, modifying and releasing Protocol Data Unit (PDU) sessions, determining the protocol used for communication (IPv4, IPv6, Ethernet, etc), and selecting the User Plane Function (UPF) that will handle the user plane traffic for each session. It also manages IP address allocation for the UE and, along with the Policy Control Function (PCF), enforces policies related to session management.

Policy Control Function

Equivalent to EPC's Policy and Charging Rules Function (PCRF), the PCF is responsible for policy control and decision-making within the 5G Core network. Besides this, the PCF also has the additional responsibility of providing policy information to the UE, such as Access Network Discovery, Selection Policies and UE Route Selection Policies.

Unified Data Management & Unified Data Repository

The Unified Data Management (UDM), along with the Unified Data Repository (UDR), serve as the 5GC equivalent to EPC's Home Subscriber Server (HSS). These functions are responsible for storing and managing subscriber data and profiles. The UDR stores user subscription information, including authentication credentials, service entitlements, and policy data. The UDM serves as a front-end to the registry, interacting with other network functions, specifically with the PCF and NEF to facilitate user authentication, authorization, and mobility management.

Network Repository Function

The Network Repository Function is a central component of the 5G Core network that serves as a service registry and discovery function. It maintains a database of available network functions and which services they offer, allowing other network functions to dynamically discover and interact with each other, without the need for them to be configured beforehand. The NRF plays a crucial role in enabling the service-based architecture of the 5G Core, facilitating communication and coordination among various network functions.

Authentication Server Function

The Authentication Server Function (AUSF) is responsible for handling authentication procedures within the 5G Core network. It works in conjunction with the UDR/UDM to authenticate UEs attempting to access the network. The AUSF is also responsible for providing security parameters in roaming contexts and ensuring that authentication processes comply with 5G security standards.

Network Exposure Function

Equivalent to EPC's Service Capability Exposure Function (SCEF), the Network Exposure Function (NEF) acts as an intermediary between the 5G Core network and external application functions. It exposes network services and events, such as the location of an UE, the reachability of the UE, its roaming status and loss of connectivity can be exposed by the NEF to authorized applications and functions, enabling them to interact with the network in a secure and controlled manner. The NEF also handles API management, security, and policy enforcement for these interactions.

Other more specific control plane functions, such as the Network Slice Selection Function (NSSF), Network Data Analytics Function (NWDAF), Non-3GPP Interworking Function (N3IWF), Application Function (AF), Short Message Service Function (SMSF) and Location Management Function (LMF) are defined.

2.4.1.2 User Plane

The user plane in the 5G Core network is composed of a single main function, the User Plane Function. It is responsible for handling all the user data traffic between the RAN and external data networks. It performs packet routing and forwarding, QoS enforcement, traffic shaping, as well as usage reporting and, in some cases, lawful interception. Multiple UPFs can be deployed in series, co-existing within the same 5G Core network, enabling distributed architectures and facilitating scalability if necessary.

2.4.2 Core Interfaces

Information exchange inside the 5G Core happens through a common access medium using APIs but, for communication with outside components, it relies on a set of interfaces that define how UEs, the RAN and the various network functions exchange information. These interfaces are designed to support the service-based architecture of the 5G Core, communicating directly with network functions. The main interfaces, as seen on figure 2.9, are summarized in Table 2.1.

Interface	Connections	Protocol
N1	UE ↔ AMF	NAS over NG-C
N2	RAN ↔ AMF	NGAP
N3	RAN ↔ UPF	GTP-U
N4	SMF ↔ UPF	GTP-C
N6	UPF ↔ external DNs	IP-based Protocols
N9	UPFs ↔ UPFs	GTP-U

Table 2.1: 5G Core Network Interfaces [29, 32]

2.5 Existing Open-Source Software for 5G Networks

Overall, Open-Source Software (OSS) has been a growing tendency in the computer engineering field, and the telecommunications sector is no exception. The adoption of OSS for 5G networks has been driven by the need for cost-effective, flexible and customizable solutions that can adapt to the rapidly evolving landscape of mobile communications.

These tools play a crucial role in research, prototyping and, in some cases, even production deployments of 5G networks, as they allow developers to freely experiment with 5G architectures without relying on expensive proprietary solutions. The existing suite of open-source projects already spans both the 5G Core and RAN, as well as other tools for support testing, evaluating performance and validation.

2.5.1 Open-Source 5G Core Implementations

On the core side, many projects have emerged to provide an accessible way to deploy a complete 5G core network using OSS components, with varying levels of maturity. Some of the most notable projects will be described below, with a brief overview on Table 2.2:

2.5.1.1 Open5GS

Open5GS [25] is an open-source implementation of a 5G Core network, written in C language. Out-of-the-box, it provides a set of both 5G Core network functions, such as control plane functions (AMF, SMF, PCF, etc.) and the user plane function (UPF), as well as 4G EPC functions for NSA implementations and backwards compatibility. It focuses on being lightweight and easy to deploy, containing extensive documentation which guides a developer from installation to configuration and, finally, operation. Each network function in Open5GS is defined in a separate configuration file, making it straightforward to modify individual components without affecting the entire system. Open5GS is one of the most widely adopted open-source 5G Core implementations, as seen in [9, 19, 22, 33], making it a popular choice for research and prototyping of 5G networks.

2.5.1.2 Free5GC

Free5GC [13] is another open-source project that offers an implementation of the 5G Core, written in Go, which integrates well with web-services. It provides a full suite of 5G Core network functions, as according to 3GPP Release 15 [2]. Although not as popular as other deployments, Free5GC has been used in academia, as seen in [19], and has an active community that contributes to its development and maintenance.

2.5.1.3 Magma

Initially developed by Facebook, Magma [12] is a project that provides 2G, 3G, 4G and 5G Core network implementations, made open-source in 2019. Despite being relatively new, it has seen a spike in contributors and users, mainly due to being designed from scratch to be implemented in poor, rural areas, where connectivity is scarce. Within their project, they offer a full 5G SA Core network implementation, written mainly in C++, Go and Python.

While not as widely adopted as other solutions, due to its novelty and lack of extensive documentation, Magma has been used in research, as seen in [18].

2.5.1.4 OpenAirInterface Core Network

The OpenAirInterface Software Alliance, composed by major global players (such as Nokia, Vodafone or Ericsson), provides a 5G Core [5] implementation, mainly maintained by its community. It is written in C++ and provides a complete set of 5G Core network functions, based on containers, making it easy to deploy and manage. The project is well-documented, has many tutorials and examples and has an active community. It is widely used in academia and research, as seen in [9, 19, 35]

Project	Programming Language	Supported Generations	Maturity level
Open5GS	C	4G/5G	Most widely adopted open-source 5G Core implementation, with extensive documentation and community support. Used in multiple research projects.
Free5GC	Go	5G	Well-maintained project with a focus on web-services integration. Less popular than Open5GS but still used in academia.
Magma	C++, Go, Python	2G/3G/4G/5G	Newer project with a focus on rural connectivity. Growing community and adoption, but less mature than other solutions.
OAI Core	C++	4G/5G	Backed by major industry players, well-documented and widely used in academia. Container-based deployment for ease of management.

Table 2.2: Overview of different open-source 5G Core implementations.

2.5.2 Open-Source 5G RAN Implementations

On the RAN side, many open-source projects have also emerged to provide 5G RAN implementations, but two key players stand out from the rest, srsRAN and OpenAirInterface RAN, due to their maturity, extensive documentation and active communities. These projects are described below, followed by a brief comparison on Table 2.3.

2.5.2.1 srsRAN

srsRAN [27] is an open-source software suite that provides a complete implementation of the 5G RAN, as defined by 3GPP. It is written in both C and C++ languages and is designed to be modular, allowing the developer to customize both the CU and DU components of the gNodeB separately.

It supports all FR1 bands, FDD/TDD modes, all bandwidths up to 100 MHz TDD and 50 MHz FDD, 15/30 kHz subcarrier spacing, 256-QAM modulation, Network Slicing,

among other features. srsRAN integrates well with Open5GS and is widely adopted in both academia and industry, as seen in [19, 22, 33, 35].

The srsRAN project also provides 4G LTE support through its srsRAN 4G project [1], allowing for backwards compatibility with existing 4G networks.

2.5.2.2 OpenAirInterface

Along with a Core Network, the OpenAirInterface Software Alliance also provides a 5G RAN implementation [6], implementing full gNodeB and UE functionalities. Also written in C and C++, it is the most complete 5G open-source RAN stack, supporting both NSA and SA deployments, monolithic gNodeB as well as disaggregated CU/DU architectures and FR2 frequency band support. Just like srsRAN, it is widely used in academia, as seen in [9, 19].

Feature	**srsRAN**	**OpenAirInterface RAN**
Frequency Bands Supported	All FR1 (sub-6 GHz) bands	All FR1 and FR2 bands
Deployments supported	NSA and SA	NSA and SA
Duplex Mode	FDD/TDD	FDD/TDD
Sub-Carrier Spacing	15/30 kHz	15 and 30 kHz (FR1), 120 kHz (FR2)
Bandwidth (BW)	10, 20, 40, 80, 100 MHz	10, 20, 40, 80, 100 MHz
Programming Language	C & C++	C & C++
Antenna Configuration	SISO	SISO and MIMO
Documentation & Support	Extensive	Fair
Deployment Complexity	Easier and faster to deploy	Complex to deploy

Table 2.3: Comparison of the open-source 5G RAN projects, as described by Mamushiane et al. [22].

2.5.3 UERANSIM

UERANSIM [16] is not a real RAN implementation, but rather a 5G UE and RAN simulator, written in C++. It can be used to test 5G Core networks without the need for SDRs. It supports both NSA and SA deployments, and can simulate multiple UEs simultaneously.

2.6 Related Works

The deployment of 5G networks using OSS has already become an area of research, with many researchers having explored the feasibility of deploying 5G networks using open-source components. Very different approaches have been taken, from small-scale testbeds to complex environments integrated with commercial equipment. In this section, a discuss of some relevant works in this area will be presented, followed by an overview of said word on table 2.4. Most of these works were sourced from srsRAN's Research tab, made available on their website.

Tío et al. [33] focused on implementing a complete 5G SA network testbed and evaluated its performance using different UEs. Their approach consisted of using Open5GS for the 5G Core network and srsRAN for the RAN, deployed on USRP X310 and B200 SDRs. To evaluate the performance of their testbed, they used srsUE, a 5G UE implementation developed by the srsRAN project, Commercial Off-The-Shelf UEs and a Sixfab 5G HAT mounted on a Raspberry Pi.

Their results showed that the testbed was able to provide connectivity to the UEs and achieve average data rates of up to 68.6 Mbps downlink. Their testbed was also able to connect to the internet and, using Speedtest by Ookla, achieve downlink speeds upwards of 20.1 Mbps. They also performed a series of qualitative tests: package downloads through `apt`, video streaming on Youtube and video calls between UEs, with varying degrees of success.

On a similar vein to the previous work, Vázquez et al. [35] too developed a complete 5G SA testbed, named "MAQ5G". Their approach relies solely on the OpenAirInterface project, deploying both their 5G Core and RAN software on USRP X310 and B210 SDRs, with some modifications to the gNodeB scheduler's code.

Although similar, the evaluation done on this work shines a bigger light on the radio performance, measuring the throughput through different means of transmission and RAN configurations, using two COTS devices outfitted with writable SIM cards. The authors also evaluated the RAN host system's performance under different traffic loads, measuring CPU usage.

Results showed that their testbed was able to provide connectivity to the UEs and was able to achieve a maximum downlink throughput of 149 Mbps, although they also observed that, under high traffic loads, the host system's behaviour became unreliable, leading to loss of connection for the UEs, being the solution a full UE reboot.

Mamushiane et al. [22] also developed a 5G SA testbed using Open5GS and srsRAN. Their report shines a bigger light on the network development, detailing extensively their configuration decisions and the necessary steps to deploy the network successfully. They also provide a guide for troubleshooting network issues.

While not being the central topic of their work, they evaluated the performance of their testbed, presenting a gNodeB trace showing perfect conditions for the connection, with 0 to 1 packets dropped on downlink and no packets dropped on uplink.

Shifting their focus towards containerization, orchestration and the complexity of managing different software stacks, Horstmann et al. [19] developed a complete general-purpose network framework named Open5GCube, able to deploy both 5G SA and NSA deployments modularly, as well as 4G LTE and 2G GSM networks.

Their approach focuses on containerization using Docker Containers, allowing the developers to interchange between different implementations of both the 5G Core and RAN components seamlessly. To orchestrate the deployment of the network, they developed their own orchestration script.

Their testbed implements Open5GS, Free5GC and OAI Core as 5G Cores, and srsRAN and OpenAirInterface RAN as RAN implementations, built upon two USRP X310 SDRs using a 10 MHz GPSDO for synchronization.

To evaluate their framework, they used several Commercial Off-The-Shelf UEs, with different Operating Systems and network chips, and a Quectel RM500Q-GL modem. Results showed that their framework was able to provide 2G and 4G connectivity to all UEs, while 5G performance varied between the different UEs , with some being able to establish connection seamlessly to the network, while others could only connect to the RAN but not initiate a PDU session.

Integrating CD/CI and automation tools, Bonati et al. [9] presented "5G-CT",a framework focused on automated deployment and Over-The-Air (OTA) testing. Their network

consisted of a Open5GS core, a A5G Networks core and OpenAirInterface RAN, deployed at a Northeastern University lab.

Their work stands out due to the tools they employed: OpenShift, a Kubernetes cluster where the Core and RAN components would be deployed, and GitOps workflows, the automation tool used to orchestrate the deployment of the network, By using these tools, they automated the provisioning of their 5G networks, effectively streamlining the setup and guaranteeing a consistent and reproducible deployment process.

To demonstrate the capabilities of their framework, they conducted a series of tests over the course of nine months. During this period, the framework executed a daily operation cycle, automatically deploying the network, connecting the UE to the network and exchanging UDP traffic between them for 6 hours at varying data rates. In the end, the framework would report the results to another OpenShift pod, where they would be analyzed, and tear down the network.

Results showed that their framework was able to setup the network successfully and tests were ran as planned, presenting little to no packet-loss.

While these works focus on OSS implementations using SDRs, Zu et al. [37] bridged the gap between OSS and commercial applications, focusing on the realism and the scale of real-world deployments. Their approach consisted on implementing a 5G core using Open5GS along with a proprietary Ericsson RAN, deployed on a 30km wireless lab in City of Ames, Iowa.

Their lab was composed of 4 gNodeBs and 30+ UE sites equipped with Quectel radios, scattered across residential areas, commercial buildings and farm lands, providing a diverse range of environments to test the network's performance. Results show that their network was able to provide connectivity to all UEs and achieve throughput speeds around 320 Mbps, thus proving that an open-source 5G Core can reliably sustain carrier-grade performance and real-world deployments.

Paper	Core	RAN	UE	Containerization	Antenna Config	SDR
[9]	OAI, Open5GS, A5G	OAI	Sierra Wireless Radio	OpenShift and GitOps workflow		8x USRP X310/X410
[19]	Open5GS, Free5GC, srsRAN, OAI	OAI, srsRAN	COTS, Modem	Docker Containers		USRP X310
[22]	Open5GS	srsRAN	COTS, Emulator	OpenStack on Docker Containers	MIMO	NI-2944R
[33]	Open5GS	srsRAN	COTS, 5G HAT, Emulator	Docker Containers	MIMO	USRP X310/B200
[35]	OAI	OAI	COTS	Docker Containers	SISO (custom scheduler)	USRP X310/B210
[37]	Open5GS	Ericsson	Quectel modem	N/A	Massive MIMO	Ericsson proprietary

Table 2.4: Overview of the network implementations of related works.

3

State of the Art

Not finished, remember to use Dissertation Plan presentation as a basis for this chapter.

4

Proposed Architecture

This chapter revolves around the proposed architecture for implementing the 5G network, using free open-source software and SDR hardware. This chapter is structured as follows: Section 4.1 details the physical hardware assets, software-defined radios, clock synchronization equipment, and user equipment. Section 4.2 describes the architectural implementation of the 5G Core Network, Section 4.3 outlines the structure of the Radio Access Network, and Section 4.4 illustrates the comprehensive system topology and the standardized interfaces bridging the entire environment, providing a logical transition into the physical implementation detailed in Chapter 5.

4.1 Hardware Components

The machines chosen for this project are two Dell Optiplex Micro 7080 machines, with the following specifications:

Component	**Machine 1**	**Machine 2**
CPU	Intel Core i7-14700T	Intel Core i7-14700T
RAM	16 GB DDR5	16 GB DDR5
Operating System	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Storage	512 GB SSD	512 GB SSD

Table 4.1: Specifications of the machines used for the 5G network setup.

The machines are connected to each other using an Ethernet cable, assigning static IP addresses to each one. Machine 1 is assigned the IP address space 10.0.0.10/24, while Machine 2 is assigned the IP address space 10.0.0.20/24. This setup allows for direct

communication between the two machines, which is essential for the operation of the 5G network testbed.

For the radio, two SDRs were chosen:

- National Instruments USRP B210: This is a mid-level SDR that uses UHD, the gold standard for SDR development. These boards are widely used in research projects and testbeds, and they are natively compatible with most SDR software.

- Nuand bladeRF xA4: This is a cheaper mid-level SDR that uses a proprietary library, libbladeRF, for communication with the host machine. It is less used than the USRP B210 in research projects, as it is not directly compatible with most software. Through the use of a wrapper library, it is possible to use this SDR with most software which don't natively support it.

Both of these boards use an Analog Devices AD9361 transceiver, which makes their RF capabilities theoretically similar. The main difference between them are their Field-Programmable Gate Arrays (FPGAs), in which the B210 uses a Xilinx Spartan-6, while the bladeRF xA4 uses an Intel Altera Cyclone V.

Along with the SDRs, a Leo Bodnar GPS Disciplined Oscillator (GPSDO) was chosen to provide a stable clock source for the radios, which is crucial for the proper operation of the 5G network, given the strict timing requirements of the 5G standard, and TDD operation. The GPSDO provides a stable 10 MHz reference clock and a 1PPS signal, which are used to synchronize the SDRs and ensure accurate timing for the transmission and reception of signals.

4.2 Core

The core network is implemented using Open5GS [25], an open-source project that provides a complete 5G core network implementation, following the 3GPP Release 17 specifications. Open5GS is designed to be modular, with each network function individualized and implemented as a separate process, each one with a different configuration file on the machine. The core network is deployed on Machine 1.

4.3 RAN

The RAN is implemented using the OCUDU Project [27] formerly known as srsRAN, an open-source software suite that provides a complete 5G NR, following the 3GPP Release 17 specifications. Written in both C and C++ languages, it is modular by design, allowing the developer to customize both the CU and DU components of the gNodeB separately. The RAN is deployed on Machine 2, and connects to the SDR via USB 3.0.

4.4 UEs

To connect to the network, two smartphones were used. The first one is a Motorola Moto G56 5G, which is a mid-range smartphone that supports 5G connectivity. The second one is a Xiaomi Redmi Note 14 5G, which is also a mid-range smartphone with 5G capabilities. Both smartphones were chosen due to their similar hardware specifications, eSIM and 5G support, but also because they have differing chipsets, the Moto G56 uses a MediaTek Dimensity 7060, while the Redmi Note 14 uses a Qualcomm Snapdragon. This allows for testing the compatibility of the network with different hardware, which is crucial for ensuring that the network can support a wide range of devices. Both smartphones are connected to the network using eSIM profiles, which are provisioned through a remote SIM provisioning service.

5

Implementation

This chapter documents the practical implementation of the testbed described in Chapter 4 and the methodology used to evaluate it. This chapter is structured as follows: Section 5.1 details the actual implementation of the testbed, the hardware and software used, and the configuration of the network. Section 5.2 describes the methodology used to evaluate the performance of the network, including the test locations, traffic conditions, and test rounds. The results of these tests will follow in Chapter 6.

5.1 Actual Implementation

Chapter 4 proposed a two-machine deployment, with one machine hosting the core network and the other hosting the RAN. In practice, the lab deployment used four machines instead: one running the 5G RAN, another running the 4G RAN, another machine running the core network (serving as a core for both generations), and a fourth reserved for a 5G NSA deployment, which was ultimately not implemented. Each working RAN was kept on its own dedicated machine rather than sharing a single host between radio access technologies, so that the 4G and 5G networks could run independently of one another rather than being intermingled on the same machine. This choice follows directly from one of the goals of this dissertation: to leave a working network testbed in place, rather than a single disposable experiment, for future projects to build on and explore.

All 4 machines are Dell Optiplex Micro 7080 machines, with the same specifications as those listed in Table 4.1. The machines are connected to each other through an ethernet switch, with static IP addresses assigned to each one, as shown in Figure 5.1.

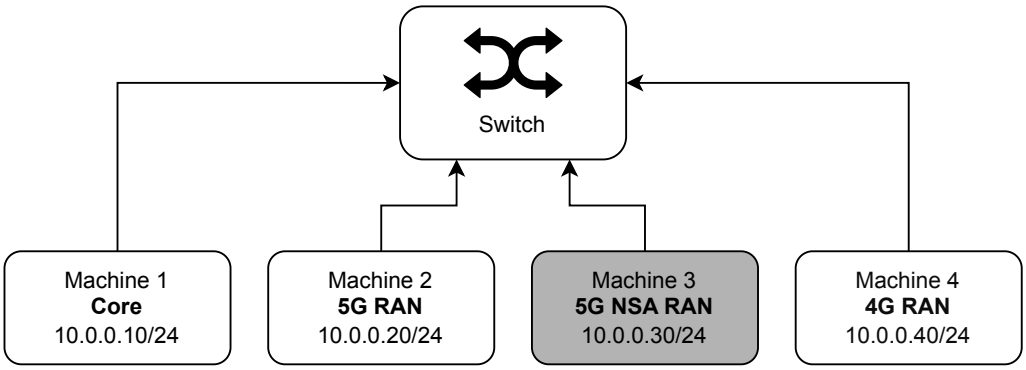


Figure 5.1: Network topology of the four machines, showing the role of each machine and its static IP address. The machine in gray was not used in the actual testbed, but was reserved for a 5G NSA deployment.

A view of the laboratory bench, showing all four machines, is shown in Figure 5.2.

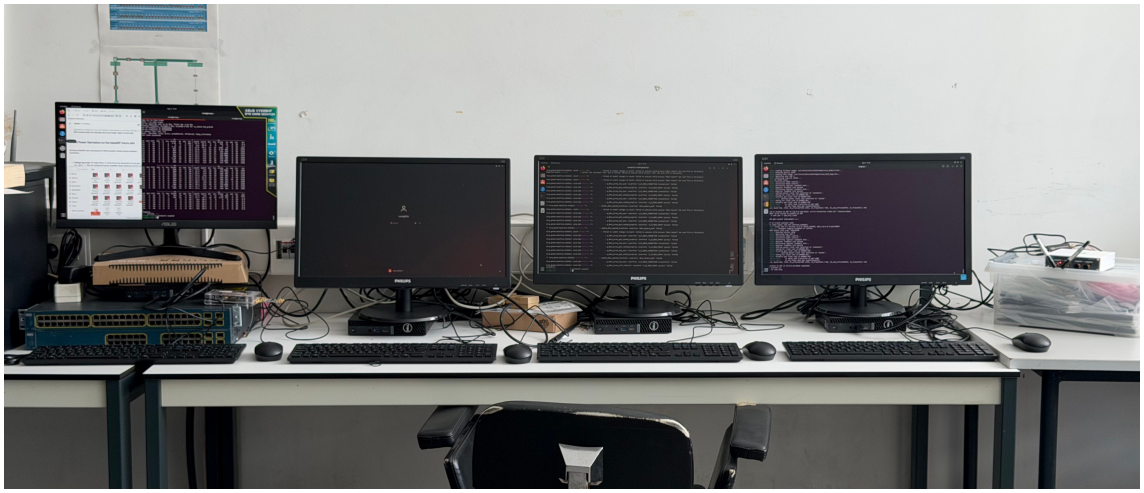


Figure 5.2: The bench, showing all 4 machines. From left to right: 4G RAN, 5G NSA (unused), Core and 5G RAN. Connected to the 4G RAN machine is the bladeRF xA4, and on the rightmost side, connected to the 5G RAN machine, is the NI B2901.

Connected to the 4G RAN machine is the Nuand bladeRF xA4 (Figure 5.3), which was used for the 4G deployment. This board, unlike the B2901, was powered entirely over its USB connection to the RAN machine, with no separate power supply used.

Connected to the 5G RAN machine is a NI B2901 (Figure 5.4), the National Instruments designation for the USRP B210 used in the proposed architecture. In addition to its USB connection to the RAN machine, the B2901 was connected to a wall socket through its own power supply, which was required to sustain the board at the gain settings used

Figure 5.3: The Nuand bladeRF xA4, connected and cabled on the bench.

Figure 5.4: The B2901 and the Leo Bodnar GPSDO, as connected on the bench.

during testing. Along with the board, the Leo Bodnar GPSDO was connected to provide both a stable clock source and a 1PPS signal for synchronization. The GPSDO's GPS lock was verified using `lbgpsdo` [17], an open-source command-line utility for configuring and querying Leo Bodnar GPSDO devices.

For both radios, the antennas used were the same. The 4G rig used a BT-200 LNA on

the RX port and a BT-100 power amplifier on the TX port, both Nuand components, seen in Figure 5.5. The 5G rig had no additional RF front-end components.

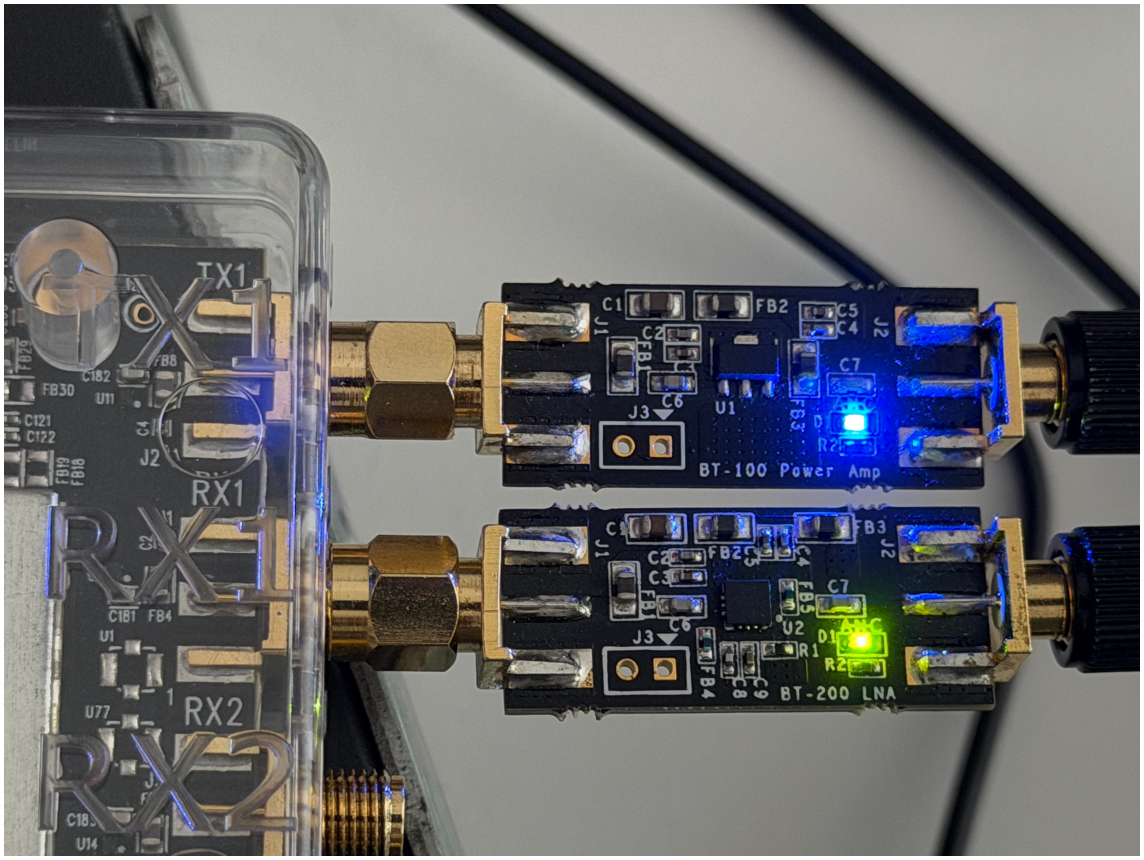


Figure 5.5: The BT-200 LNA and BT-100 power amplifier bias tees, mounted on the bladeRF xA4's RX1 and TX1 ports respectively.

For the UEs, the initial proposed architecture (Chapter 4) listed a Motorola Moto G56 5G. Additionally, a Samsung Galaxy Z Flip 5 (personal device) was brought in, and became the primary UE for the actual test campaigns, as during the tests, the Moto G56 struggled to connect to the network. The cause of this issue was never identified, and no speculation is made here.

Device	Chipset	RAM / Storage
Motorola Moto G56 5G [14]	MediaTek Dimensity 7060	8 GB / 512 GB
Samsung Galaxy Z Flip5 [15]	Qualcomm Snapdragon 8 Gen 2	8 GB / 512 GB

Table 5.1: Comparison of the two candidate UEs. Both support NR band n78, the band used by this testbed.

On the software side, the 5G OCUDU RAN [27] ran at commit 29dc1aede7, with the

configuration present in Table 5.2.

Parameter	Value
Band	n78 (TDD)
Downlink/Uplink centre frequency	3410 MHz
`dl_arfcn`	627340
Channel Bandwidth	20 MHz
Subcarrier Spacing	30 kHz
PLMN	00101
TAC	7
PCI	1
`tx_gain`	89.75

Table 5.2: Static configuration for the 5G OCUDU RAN.

The parameter `rx_gain` and the TDD structure were changed between testing rounds, and will be discussed further upon in Section 5.2.

The 4G srsRAN RAN [1] ran at commit 6bcbd9e5b, with no configuration changes across testing rounds. The configuration parameters are presented in Table 5.3.

Parameter	Value
Band	7 (FDD)
Downlink centre frequency	2680.0 MHz
Uplink centre frequency	2560.0 MHz
Channel Bandwidth	10 MHz (50 PRB)
PCI	1
PLMN	00101
TAC	7
Cell ID	1
`tx_gain`	66
`rx_gain`	60

Table 5.3: Static configuration for the 4G srsRAN RAN. No parameters changed across testing rounds.

Finally, the Open5GS [25] core ran at version 2.4.6. Harking back to the reusability objective stated in the Introduction, all configuration files for the core were made available in a GitHub repository [28] and a script to deploy them directly onto the core machine was created and made public, allowing anyone to redeploy the same core configuration without having to reconstruct it by hand. The script, `deploy_open5gs_configs.sh`, sparse-checks out the configuration files from the project's GitHub repository directly onto the core host. This approach ensures that the testbed's configuration can be easily replicated in the future,

supporting the goal of reusability and facilitating further research or experimentation based on this work.

5.2 Test Methodology

The network's performance was evaluated at three fixed UE positions in the same room as the RAN, chosen to combine increasing distance from the radio with different obstruction condition at each point. Figure 5.6 shows the top plan view of the room, and Figure 5.7 shows the elevation, giving the height of the radio and of the UE at each location above the floor.

Material and thickness are recorded for each obstruction as they, not distance alone, account for the difference between the three locations: signal attenuation through a wall depends strongly on its material and thickness, particularly at the 3.4 GHz centre frequency used by the 5G RAN, so the three locations should be read as distance-plus-obstruction conditions rather than a simple distance variation.

Besides obsctructions, the objects present in the room may account for signal attenuation as well. In Figure 5.6, the objects shown in dark grey are metal furniture and equipment present in the room. None of these lie directly on the signal path between the RAN and any of the three test locations, but their metal construction means they may still have contributed to reflections and multipath propagation, and should be kept in mind when interpreting the results. Other objects present in the room, such as the tables and doors presented on the diagram, present minimal attenuation and are not expected to have a significant effect on the results. A summary of all measurements and signal obstruction conditions for each of the three test locations is presented in Table 5.4.

Location	Distance	Height	Obstruction	Material	Thickness
A	2 m	75 cm	Clear line of sight	—	—
B	6 m	72 cm	Office partition	Plastic and Felt	3 cm
C	10 m	59 cm	Concrete wall	Concrete	25.3 cm

Table 5.4: The three test locations: distance from the radio, UE height above the floor, the obstruction material and thickness on the direct path at each point. All distances, thicknesses and heights were measured directly with a tape measure.

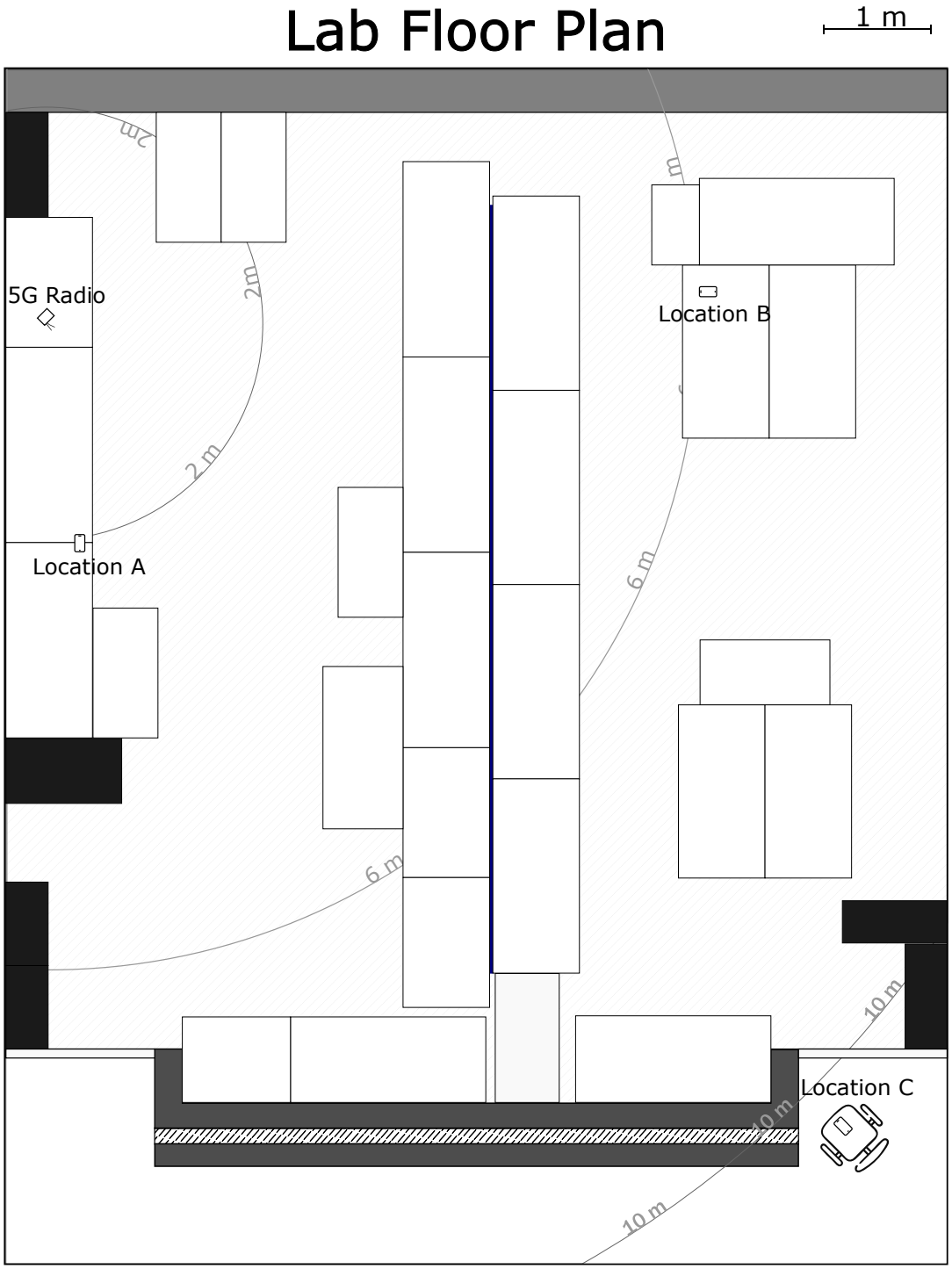


Figure 5.6: Plan view of the room, showing the radio position and the three UE test locations, drawn to scale.

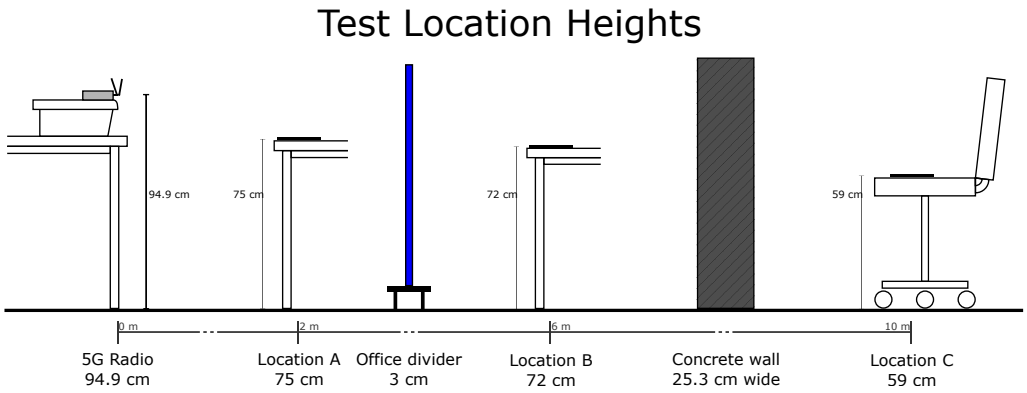


Figure 5.7: Elevation view showing the mounting height of the radio and the UE height at each test location, along with the obstruction on the path to locations B and C.

Figure 5.8: Tape-measure thickness readings for the two obstructions on the direct path: the concrete wall at Location C (25.3 cm) and the office partition at Location B (3 cm).

At each location, tests were run for each generation of network, for both TCP and UDP, and for both uplink and downlink, giving four traffic conditions per location, per generation. For the 4G network, each of these conditions was tested once (n=1); for the 5G network, each was repeated three times (n=3). This difference was not a deliberate methodological choice about sample size per generation: the 4G network was only tested during the first round of testing, in which every combination, both in 4G and 5G, was only tested once. Subsequent rounds of testing focused exclusively on the 5G network, following configuration corrections described later in this chapter, and used three repetitions per condition from that point onward; the 4G network was deliberately left untouched throughout, so its results were never repeated.

Every test consisted of an `iperf3` run lasting 20 seconds with 8 parallel streams, with the same parameters used across all tests regardless of generation, location, protocol, or direction. On the UE side, these tests were run using the CellularLab app [4]; the `iperf3`

server was hosted on the core machine.

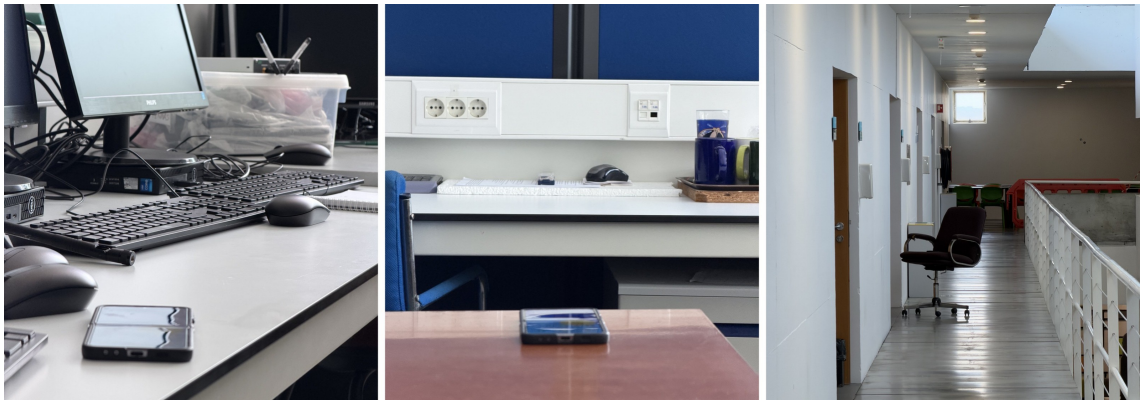


Figure 5.9: The three test locations, with the UE in place: (a) Location A, 2 m, clear line of sight; (b) Location B, 6 m, behind the office partition; (c) Location C, 10 m, behind the concrete wall.

Initially, the 5G network was configured with a 3:1 TDD structure. This structure is set through the `tdd_ul_dl_cfg` block of the RAN configuration, shown in Listing 5.1.

Listing 5.1: `gnb.yaml` TDD configuration for the 3:1 pattern used in Round 1.

```
tdd_ul_dl_cfg:
  dl_ul_tx_period: 5
  nof_dl_slots: 3
  nof_dl_symbols: 6
  nof_ul_slots: 1
  nof_ul_symbols: 2
```

This configuration dictates that for every 5-slot period, 3 slots are allocated for downlink and 1 slot is allocated for uplink, with the remaining flexible slot composition being: 6 symbols for downlink, 2 symbols for uplink and the remainder (6 symbols) are flexible. In total, given that a slot has 14 symbols, the total number of symbols per period is 70, with 48 symbols (3 * 14 symbols + 6 symbols) allocated for downlink and 16 symbols (1 * 14 symbols + 2 symbols) allocated for uplink. That gives 68.6% of airtime for downlink and 22.9% for uplink per period of transmission, with the remaining airtime being guard period.

Figure 5.10 shows the slot and symbol structure of the 3:1 TDD pattern, which was used in Round 1.

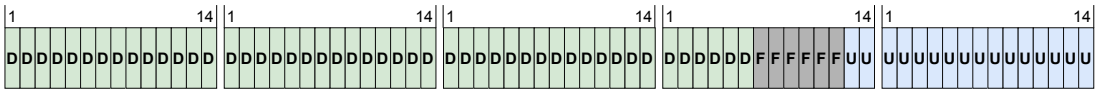


Figure 5.10: Slot and symbol structure of the 3:1 TDD pattern used in Round 1.

This structure was later found to impose a hard structural cap on uplink airtime, independent of signal quality, and was rebalanced to a symmetrical 2:2 structure from Round 2 onward, which meant that each transmission period was composed of two full downlink slots and two full uplink slots, along with a symmetrical flexible slot. Listing 5.2 shows the corresponding `tdd_ul_dl_cfg` block.

Listing 5.2: `gnb.yaml` TDD configuration for the rebalanced 2:2 pattern used from Round 2 onward.

```yaml
tdd_ul_dl_cfg:
  dl_ul_tx_period: 5
  nof_dl_slots: 2
  nof_dl_symbols: 5
  nof_ul_slots: 2
  nof_ul_symbols: 5
```

With this configuration, out of the 70 total symbols per period, 33 symbols (2 * 14 symbols + 5 symbols) are equally assigned to both downlink and uplink, giving 47.1% of airtime for each direction, with the remaining airtime being guard period.

Figure 5.11 shows the resulting slot and symbol structure.

Figure 5.11: Slot and symbol structure of the rebalanced 2:2 TDD pattern used from Round 2 onward.

All together, the rebalance is expected to roughly double the uplink airtime available to the scheduler (22.9% $\rightarrow$ 47.1% = $\times$2.06) while reducing the downlink share to little more than two-thirds of what it was under the 3:1 pattern (68.6% $\rightarrow$ 47.1% = $\times$0.69), trading some downlink capacity for a substantial gain in uplink capacity. Whether this

expected trade-off actually materialised in the measured throughput, and how closely the scheduler's grant rates tracked it, is examined in Section 6.2 of Chapter 6.

Pattern	DL slots	UL slots	DL symbols	UL symbols	DL airtime	UL airtime	
3:1 (Round 1)	3	1	48	16	68.6%	22.9%	
2:2 (Round 2 onward)	2	2	33	33	47.1%	47.1%	

Table 5.5: Slot and symbol allocation per 5-slot, 70-symbol transmission period, for the two TDD patterns used across testing.

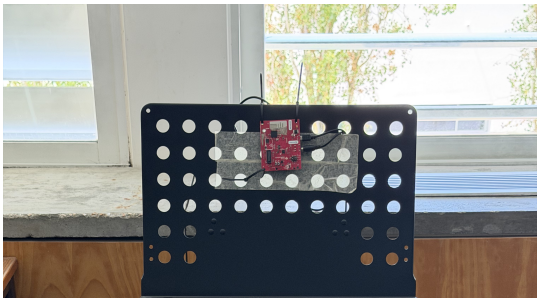
5.2.1 Test Round 1

This test round was the first to be run, on the 28th of July 2026. It comprised of 24 tests, 12 for 4G and 5G generations, differing in location, protocol and direction, each with a single repetition. The 5G network was run with the 3:1 TDD structure, remnant of the original OCUDU configuration used as a basis for the network configuration, which was later found to structurally cap the maximum uplink airtime and was later rebalanced to a symmetrical 2:2 structure for Round 2. Running a single test for each configuration was also found to produce insufficient analytical data, thus not capturing the real behavior of the 5G network, so the following rounds were run with three repetitions per configuration. Results from this round are presented in Chapter 6.

5.2.2 Test Round 2

This test round was run on the 2nd of August. The 5G network was reconfigured with the balanced 2:2 TDD structure devised after Round 1, and testing proceeded with three repetitions per configuration at each location; the 4G network was not re-tested. The TDD rebalance alone resolved the uplink problem from Round 1, with 5G uplink throughput matching the 4G baseline at 2 m and 10 m, though the downlink regressed as a trade-off for the additional uplink airtime. Location B, at 6 m, produced unusable data this round, with the UE re-attaching to the network 18 times during the session, and was excluded from analysis.

A likely, though unconfirmed, explanation was only identified afterward: a full metal stand (Figure 5.12), belonging to an unrelated



thesis project sharing the lab, was standing directly in the signal path between the UE and RAN at this location and was unnoticed at the time. Despite the nominal line-of-sight condition, a metal obstruction of this size may have been enough to block the link given the radio's otherwise limited transmit power. Keeping `rx_gain` at 70 for this round was later found to be overloading the receiver at 2 m, and measures were taken to address this issue on round 3. Results from this round are also presented in Chapter 6.

5.2.3 Test Round 3

Before this round, the metal stand identified as a likely obstruction at Location B was moved out of the signal path, and a gain sweep was run at Location A to determine the optimal `rx_gain` setting for the 5G network, resulting in a locked-in value of 60. This test round was run on the 3rd of August, with the 5G network reconfigured to this `rx_gain` of 60, keeping the 2:2 TDD structure from Round 2, and with three repetitions per configuration; once again, the 4G network was not re-tested. This setting resolved the receiver overload found in Round 2, more than doubling 5G uplink throughput over the 4G baseline at 2 m, but severely limited uplink performance at 10 m, where throughput dropped to roughly a sixth of what `rx_gain` 70 had achieved at the same location in Round 2. This showed that a fixed, manually-set receiver gain cannot serve every point in the cell. Results from this round are presented in Chapter 6.

Table 5.6 summarises the 5G configuration and data validity across the three rounds for reference; the reasoning behind each change is described above.

Round	Date	TDD	rx_gain	Reps	4G tested	Total reps	Locations valid
1	28 Jul	3:1	70	1	Yes	24	A, B, C
2	2 Aug	2:2	70	3	No	36	A, C (B excluded)
3	3 Aug	2:2	60	3	No	36	A, B, C

Table 5.6: Summary of the 5G network configuration and data validity across the three test rounds. Round 1 covers both generations; Rounds 2 and 3 cover the 5G network only, as the 4G network was not re-tested. Round 1's 5G and 4G tests were captured on different dates, 25 and 28 July respectively.

Together, these three rounds make up the full test campaign analysed in Chapter 6.

6

Results

This chapter presents the results of the performance evaluation of the 4G and 5G networks implemented in the testbed described in Chapter 5.

6.1 General comparison between 4G and 5G

This section presents a comparison of the 4G baseline against the 5G network across the three test rounds described in Section 5.2, focusing on the evolution of performance metrics. Because the 5G configuration changed between rounds, namely the TDD structure and the `rx_gain` setting, these results are presented as a progression rather than a single snapshot, tracking how each change affected throughput at each location and in each direction. As will become clear over the following sections, no single round can be presented as the definitive 5G result: the configuration that performs best at short range is not the same as the one that performs best at long range, a trade-off examined in more detail later in this chapter.

In Figures 6.1 through 6.4, the mean throughput results for TCP and UDP in both downlink and uplink directions are presented for each test location, with black lines representing the range between Min/Max throughput values between each repetition for that combination. These figures mainly illustrate the performance differences between 4G and 5G across the three rounds of testing, with the results showing that the 5G network can outperform the 4G baseline in certain configurations and locations, but also underperform in others.

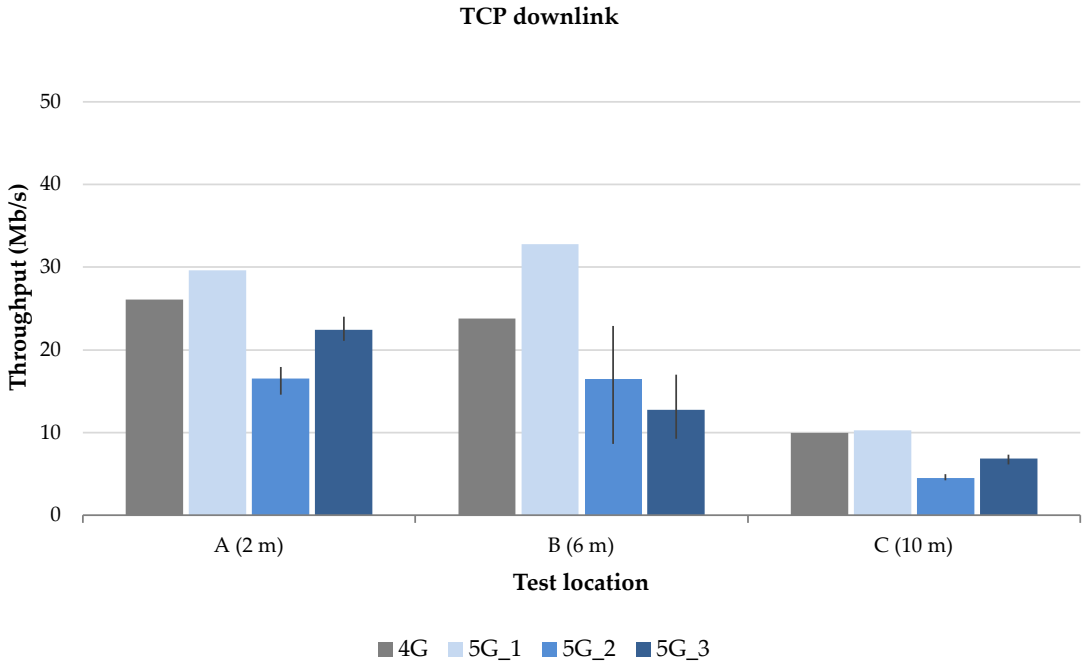


Figure 6.1: TCP downlink throughput at each test location.

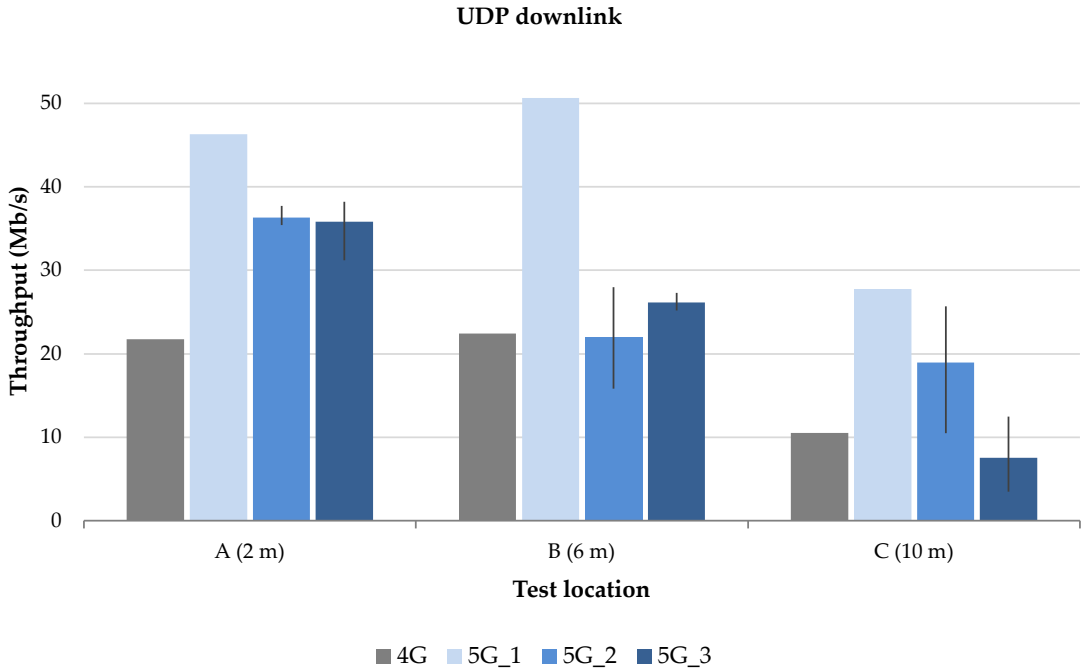


Figure 6.2: UDP downlink throughput at each test location.

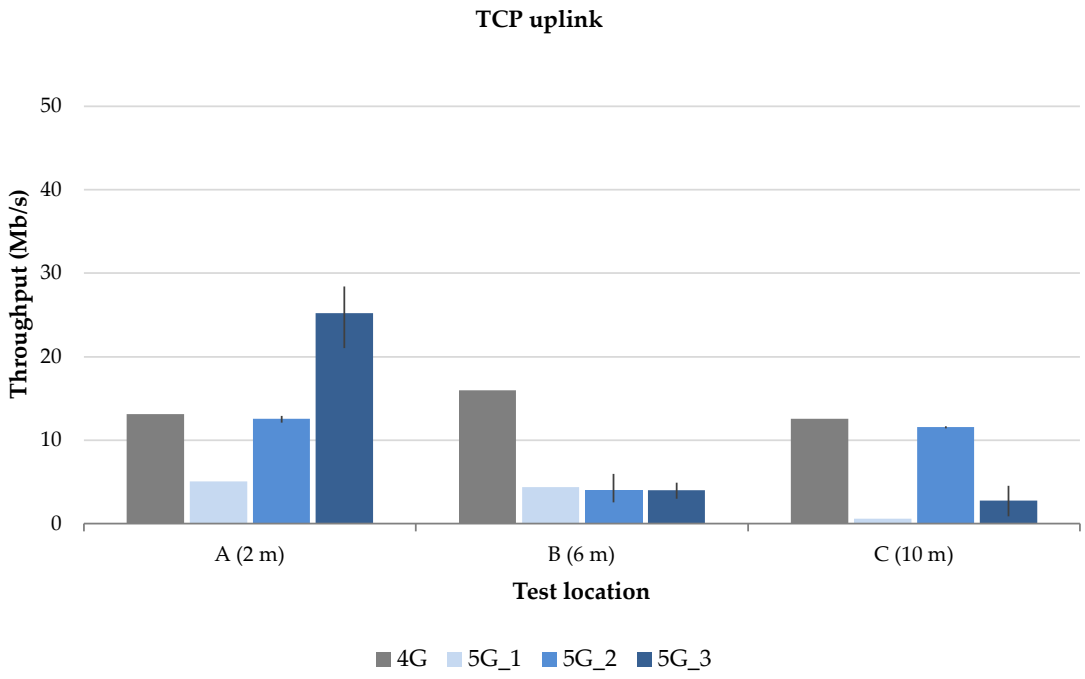


Figure 6.3: TCP uplink throughput at each test location.

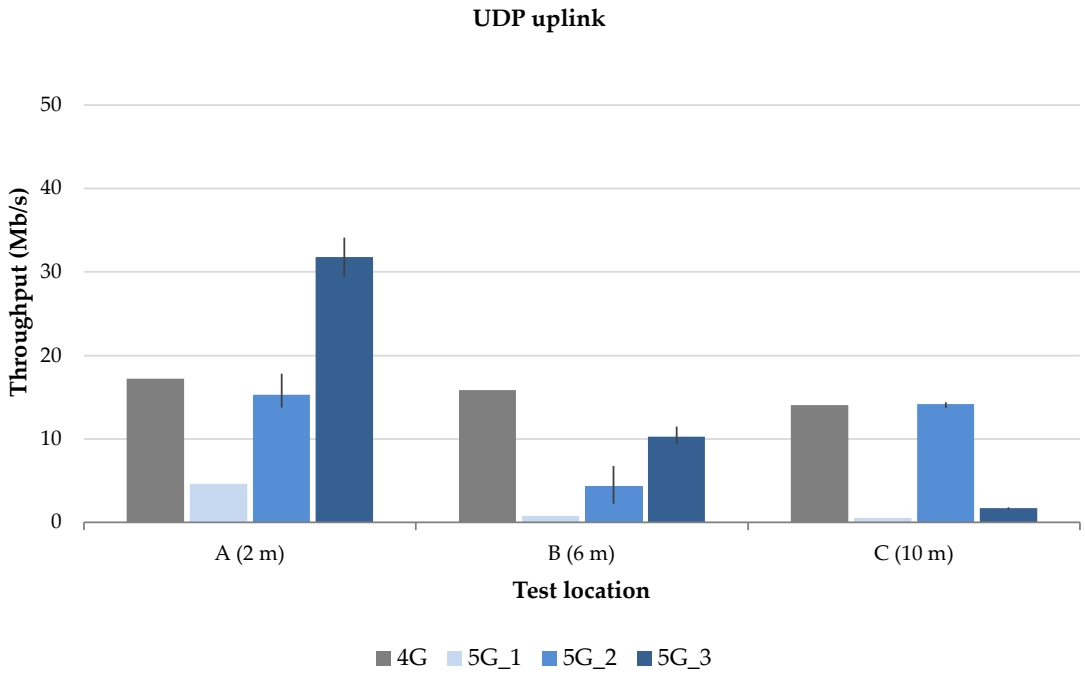


Figure 6.4: UDP uplink throughput at each test location.

6.2 Uplink airtime and TDD decomposition

One of the troubles identified in the first round of testing was that the 5G uplink throughput was capped by the TDD structure configuration implemented in the testbed, which, as mentioned in Section 5.2, allocated TDD timeslots disproportionately in favor of the downlink. The figures in the section show that with this configuration, the 5G downlink reigned with 68.8% of the total airtime, compared with 22.9% for the uplink.

We can verify this asymmetry by examining the raw metrics from the gNB traces, checking the scheduler's actual uplink grant rate against the theoretical maximum allowed by the TDD configuration.

On the radio configuration file, shown in Listing 6.1, the `cell_cfg` block sets the Subcarrier Spacing (SCS) to 30 kHz. As explained in Chapter 2, at this SCS, a slot is 0.5 ms long, and the 3:1 TDD pattern sets a 5-slot transmission period with one uplink slot, giving a total period of 2.5 ms. This means that the maximum number of uplink slots that can be scheduled per second is 1000 ms/2.5 ms = 400 uplink slots, which is the hard ceiling for uplink transmissions per second, regardless of the radio link quality.

Listing 6.1: `gnb.yaml` TDD configuration for the 3:1 pattern used in Round 1.

```
cell_cfg:
  dl_arfcn: 627340
  band: 78
  channel_bandwidth_MHz: 20
  common_scs: 30
  plmn: "00101"
  tac: 7
  pci: 1
```

Checking the raw metrics from one of the gNB traces (excerpt shown in Listing 6.2), we see that the `ok` and `nok` columns sum to 400 transmissions for each second, confirming that the scheduler was granting the full 400 uplink slots/s allowed by the 3:1 TDD pattern, with a small percentage of them failing to decode. This confirms that the uplink throughput was indeed capped by the TDD configuration, rather than by the radio link quality.

Listing 6.2: UL block of one line of the Round 1 `gnb.log` console-metric trace at Location A (2 m), saturated TCP uplink. The `ok` and `nok` columns are the counts of successful and failed uplink transmissions in that one-second reporting period; `pusch/rsrp` are the reported SINR and RSRP, `brate` the resulting bit rate.

```
|------------------------------UL-----------------------------
| pusch rsrp ri mcs brate ok nok (%) bsr ta phr
| 18.3 -17.0 1 14 7.77M 382 18 4% 700k 240n 3
| 19.1 -16.2 1 15 7.68M 376 24 6% 700k 227n 0
| 19.7 -15.6 1 16 6.25M 308 92 23% 700k 234n -3
```

The `ok` and `nok` columns sum to 376 + 24 = 400 transmissions for that second: exactly the ceiling derived above, confirming that the scheduler was granting the full 400 uplink slots/s the 3:1 pattern allows, with 6% of them failing decode. Averaged over the full Round 1 uplink runs at Locations A and C, this settles to 372–394 grants/s, matching the same ceiling within measurement noise. The 4G baseline, in contrast, grants one uplink subframe every millisecond, roughly 975/s — a 2.5× structural penalty that no amount of signal quality could recover, since it is imposed before a single symbol is transmitted. The second factor is the payload carried per grant: at Location A (2 m) the 5G and 4G payloads are essentially identical (13 634 vs. 13 429 bits per grant), so the two systems are on equal footing on link budget alone, but at Location C (10 m) the 5G payload per grant collapses by 8.7× while the 4G payload is unchanged. Multiplying the two factors reproduces the measured throughput deficit almost exactly: $2.5 \times 1.0 = 2.6\times$ at Location A against a measured 2.59×, and $2.5 \times 8.7 = 21.5\times$ at Location C against a measured 21.5×. It is this exact decomposition, not the raw throughput gap, that justifies treating the TDD pattern as a distinct and independently correctable factor from the link-budget problems examined in Section **??**.

On the subsequent rounds of testing, the TDD pattern was rebalanced to a symmetrical 2:2 pattern, which allowed two full downlink and uplink slots, with the flexible slot also adjusted to be symmetrical, per period. This change was expected to improve uplink throughput, at the cost of downlink throughput, making their performance almost equal, which Figure 6.5 presents. Only one configuration variable changes between each pair of rounds in the figure — the TDD pattern between rounds 1 and 2, and `rx_gain` between

rounds 2 and 3 — so the two steps should be read as evidence for two separate effects rather than a single continuous trend.

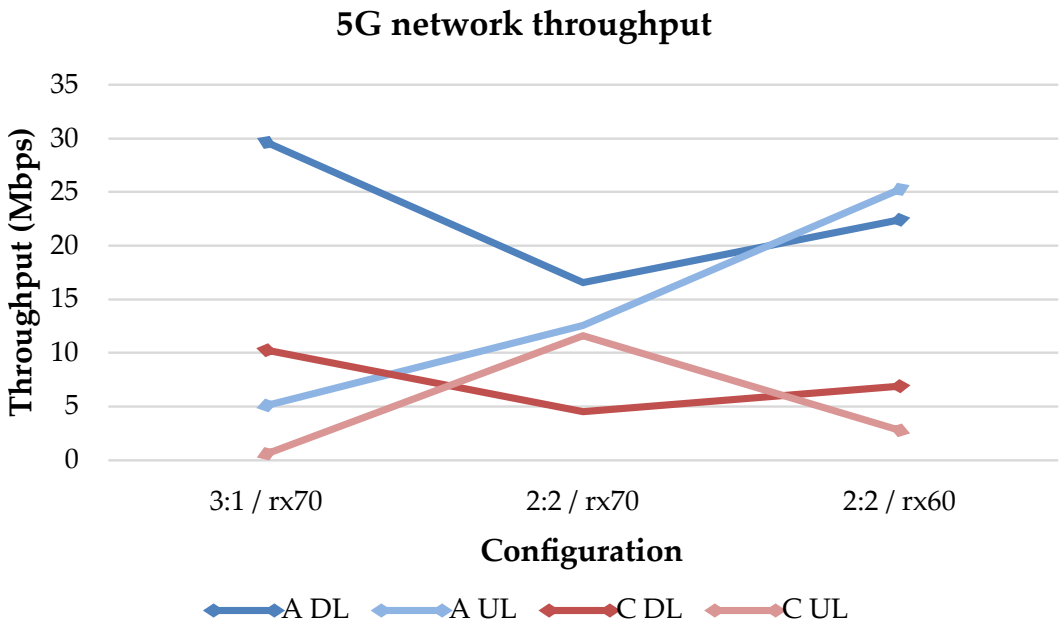


Figure 6.5: TCP throughput at Locations A (2 m) and C (10 m) across the three 5G rounds. The x-axis shows the specific configuration used in each round.

We see on the figure that initially, as expected, when using the 3:1 TDD pattern, the uplink throughput on both location A and C was significantly lower than the downlink throughput. On round 2, after rebalancing the TDD pattern to 2:2, the uplink performance rose to nearly match the downlink in location A, surpassing the downlink throughput at location C. We also note that the downlink throughput decreased in both locations, as expected.

The grant counts confirm that this rebalance delivered exactly what the airtime accounting predicted, rather than the improvement being inferred from throughput alone: the 2:2 pattern doubles the uplink ceiling to 800 slots per 2.5 ms period, and the scheduler achieves the full 800/s at both Location A and Location C. The same accounting predicts the downlink share should fall to three-quarters of its Round 1 value (×0.75); the observed downlink throughput fell further still, to ×0.56 at Location A and ×0.44 at Location C, so some factor beyond airtime alone also affected the downlink. One caveat applies to comparing Round 1 and Round 2 by throughput directly: at identical gain settings, uplink

SINR rose by 5.4 dB at Location A and by 22.4 dB at Location C between the two rounds, for reasons that remain unexplained. The raw uplink throughput gain between rounds therefore cannot be attributed to the TDD change alone — the 19.9× increase measured at Location C, for instance, far exceeds the ×2 that airtime alone predicts. The grants-per-second accounting above is unaffected by this confound, since the achieved grant rate does not depend on the SINR reported for each grant; the confound lives entirely in the bits-per-grant term.

Finally, on round 3, after adjusting the `rx_gain` to 60: at location A (2 m), the overall network throughput increased significantly, as this change decreased the number of `ovl` events on the uplink, thus stabilizing the network's performance. However, at location C, further away from the radio, the uplink throughput decreased significantly as well, due to the lower `rx_gain` desensitizing the receiver and reducing the link budget. This change was expected as a trade-off between the two locations, and it highlights the challenge of finding a single configuration that optimizes performance across all distances.

We also see that in this final configuration, the throughput between both directions is closest to being equal at short range: at Location A the two directions converge to within a narrow margin (22.4 vs. 25.2 Mb/s), the best balance observed anywhere in the dataset, while at Location C the `rx_gain` reduction gives back most of the uplink gain Round 2 had recovered, swinging the downlink-to-uplink ratio past parity to 2.48. This Location C behaviour is not a contradiction of the TDD result but the `rx_gain` trade-off that Section **??** examines in detail.

Bibliography

[1] S. R. S. (SRS). *srsRAN: Open Source 4G Software Radio Suite*. URL: https://github.com/srsran/srsran_4g (cit. on pp. 2, 22, 35).

[2] 3rd Generation Partnership Project (3GPP). *Release 15*. https://www.3gpp.org/specifications-technologies/releases/release-15. Accessed: 2026-01-27. 2018 (cit. on pp. 9, 20).

[3] 3rd Generation Partnership Project (3GPP). *Release 17*. https://www.3gpp.org/specifications-technologies/releases/release-17. Accessed: 2026-01-27. 2022 (cit. on p. 14).

[4] Abhi5h3k. *CellularLab: iPerf3 Client for Android*. URL: https://github.com/Abhi5h3k/CellularLab (cit. on p. 38).

[5] O. S. Alliance. *oai-cn5g-fed: Federation of the OpenAir CN 5G Repositories*. URL: https://gitlab.eurecom.fr/oai/cn5g/oai-cn5g-fed (cit. on p. 21).

[6] O. S. Alliance. *OpenAirInterface5G: Open Source 4G/5G RAN and Core Network*. URL: https://gitlab.eurecom.fr/oai/openairinterface5g (cit. on pp. 2, 22).

[7] J. Ansari et al. "Performance of 5G Trials for Industrial Automation". In: *Electronics* 11.3 (2022). ISSN: 2079-9292. DOI: 10.3390/electronics11030412 (cit. on p. 1).

[8] M. Barbosa et al. "Open-Source 5G Core Platforms: A Low-Cost Solution and Performance Evaluation". In: *2025 International Conference on Information Networking (ICOIN)*. 2025. DOI: 10.1109/ICOIN63865.2025.10992769 (cit. on p. 2).

[9] L. Bonati et al. "5G-CT: Automated Deployment and Over-the-Air Testing of End-to-End Open Radio Access Networks". In: *IEEE Communications Magazine* 63.1 (2025), pp. 155–162. DOI: 10.1109/MCOM.001.2300675 (cit. on pp. 20–22, 24, 26).

[10] G. Brown. *Ultra-Reliable Low-Latency 5G for Industrial Automation. A Heavy Reading White Paper Produced for Qualcomm Inc.* White Paper. Accessed: 2026-01-20. Heavy Reading, 2018. URL: https://www.qualcomm.com (cit. on p. 1).

[11] W. Ejaz et al. "Internet of Things (IoT) in 5G Wireless Communications". In: *IEEE Access* 4 (2016), pp. 10310–10314. DOI: 10.1109/ACCESS.2016.2646120 (cit. on p. 1).

[12] Facebook. *Magma: Open Source Software Platform for Building Carrier-Grade Networks*. URL: https://github.com/magma/magma (cit. on p. 20).

[13] *free5GC: Open Source 5G Core Network Based on 3GPP Release 15*. URL: https://github.com/free5gc/free5gc (cit. on pp. 2, 20).

[14] GSMArena. *Motorola Moto G56 5G – Full Phone Specifications*. URL: https://www.gsmarena.com/motorola_moto_g56_5g-13841.php (cit. on p. 34).

[15] GSMArena. *Samsung Galaxy Z Flip5 – Full Phone Specifications*. URL: https://www.gsmarena.com/samsung_galaxy_z_flip5-12252.php (cit. on p. 34).

[16] A. Güngör. *UERANSIM: Open Source 5G UE and RAN (gNodeB) Simulator*. URL: https://github.com/aligungr/UERANSIM (cit. on p. 22).

[17] hamarituc. *lbgpsdo: Configuration Utility for Leo Bodnar GPSDO*. URL: https://github.com/hamarituc/lbgpsdo (cit. on p. 33).

[18] S. Hasan et al. "Building Flexible, Low-Cost Wireless Access Networks with Magma". In: *GetMobile: Mobile Comp. and Comm.* 27.3 (2023-11), pp. 40–47. ISSN: 2375-0529. DOI: 10.1145/3631588.3631599 (cit. on pp. 2, 20).

[19] T. Horstmann et al. *open5Gcube: A Modular and Usable Framework for Mobile Network Laboratories*. 2025. arXiv: 2505.14501 [cs.NI] (cit. on pp. 20–22, 24, 26).

[20] K. Khujamatov et al. "Existing Technologies and Solutions in 5G-Enabled IoT for Industrial Automation". In: *Blockchain for 5G-Enabled IoT: The new wave for Industrial Automation*. Ed. by S. Tanwar. Cham: Springer International Publishing, 2021,

pp. 181–221. ISBN: 978-3-030-67490-8. DOI: 10.1007/978-3-030-67490-8_8 (cit. on p. 1).

[21] Z. Lv, J. Lloret, and H. Song. "Internet of Things and augmented reality in the age of 5G". In: *Computer Communications* 164 (2020), pp. 158–161. ISSN: 0140-3664. DOI: https://doi.org/10.1016/j.comcom.2020.08.019 (cit. on p. 1).

[22] L. Mamushiane et al. "Deploying a Stable 5G SA Testbed Using srsRAN and Open5GS: UE Integration and Troubleshooting Towards Network Slicing". In: *2023 International Conference on Artificial Intelligence, Big Data, Computing and Data Communication Systems (icABCD)*. 2023, pp. 1–10. DOI: 10.1109/icABCD59051.2023.10220512 (cit. on pp. 2, 8, 20, 22, 24, 26).

[23] *MongoDB Community Edition: Installation Guide for Linux*. URL: https://www.mongodb.com/docs/manual/administration/install-community/?operating-system=linux&linux-distribution=ubuntu&linux-package=default&search-linux=with-search-linux (cit. on p. 64).

[24] *Open5GS Guide: Quick Start*. URL: https://open5gs.org/open5gs/docs/guide/01-quickstart/ (cit. on p. 62).

[25] *Open5GS: Open Source 5G Core Project*. AGPL-3.0 License. URL: https://github.com/open5gs/open5gs (cit. on pp. 2, 5, 20, 29, 35).

[26] M. N. Patwary et al. "The Potential Short- and Long-Term Disruptions and Transformative Impacts of 5G and Beyond Wireless Networks: Lessons Learnt From the Development of a 5G Testbed Environment". In: *IEEE Access* 8 (2020), pp. 11352–11379. DOI: 10.1109/ACCESS.2020.2964673 (cit. on p. 1).

[27] T. O. Project. *OCUDU: Open Source O-RAN 5G CU/DU Solution (formerly srsRAN Project)*. Governed under the Linux Foundation; the srsRAN_Project GitHub repository was archived in June 2026 in favour of this repository. URL: https://gitlab.com/ocudu/ocudu (cit. on pp. 2, 21, 30, 34).

[28] M. Romão. *Open-Source 5G Testbed – Thesis Repository*. URL: https://github.com/mvromao/thesis (cit. on p. 35).

[29] S. Rommer et al. *5G CORE NETWORKS*. Ed. by Elsevier. Academic Press, 2020 (cit. on pp. 16, 19).

[30] M. Rouili et al. "Evaluating Open-Source 5G SA Testbeds: Unveiling Performance Disparities in RAN Scenarios". In: *2024 IEEE/IFIP Network Operations and Management Symposium (NOMS)*. 2024, pp. 1–6. DOI: 10.1109/NOMS59830.2024.10575687 (cit. on p. 2).

[31] Y. Siriwardhana et al. "A Survey on Mobile Augmented Reality With 5G Mobile Edge Computing: Architectures, Applications, and Technical Aspects". In: *IEEE Communications Surveys & Tutorials* 23.2 (2021), pp. 1160–1192. DOI: 10.1109/COMST.2021.3061981 (cit. on p. 1).

[32] S. Sirotkin. *5G Radio Access Network Architecture: The Dark Side of 5G*. Ed. by I. Press. Wiley, 2021 (cit. on pp. 11–13, 16, 19).

[33] B. Tío et al. *Exploring 5G with Open Source Solutions: Functional Testbed and User Equipment Evaluation*. Tech. rep. Facultad de Ingeniería, Universidad de la República, 2025. URL: https://case.ar/wp-content/uploads/2025/07/Exploring-5G-with-Open-Source-Solutions.pdf (cit. on pp. 2, 20, 22, 23, 26).

[34] M. Torres Vega et al. "Immersive Interconnected Virtual and Augmented Reality: A 5G and IoT Perspective". In: *Journal of Network and Systems Management* 28.4 (2020), pp. 796–826. ISSN: 1573-7705. DOI: 10.1007/s10922-020-09545-w (cit. on p. 1).

[35] P. Vázquez et al. "MAQ5G: Deployment of a Complete 5G Standalone Network Testbed for Testing and Development". In: *2024 XVI Congreso de Tecnología, Aprendizaje y Enseñanza de la Electrónica (TAEE)*. 2024, pp. 1–7. DOI: 10.1109/TAEE59541.2024.10604930 (cit. on pp. 21–23, 26).

[36] D. Wang et al. "From IoT to 5G I-IoT: The Next Generation IoT-Based Intelligent Algorithms and 5G Technologies". In: *IEEE Communications Magazine* 56.10 (2018), pp. 114–120. DOI: 10.1109/MCOM.2018.1701310 (cit. on p. 1).

[37] G. Zu et al. "Real-World Integration and Evaluation of Open-Source 5G Core with Commercial RAN". In: *2025 IEEE Military Communications Conference (MILCOM)*.

2025-10, pp. 1296–1301. DOI: 10.1109/MILCOM64451.2025.11309851 (cit. on pp. 25, 26).

4G Network Setup

Before developing the 5G network testbed, a 4G network was first established to serve as a performance baseline and a proof-of-concept for all hardware and software components. The 4G network setup was essential for several reasons:

- Validating that the chosen hardware and software stack worked correclty
- Establishing a known and well-documented reference point for network behavior
- Identifying potential deployment hurdles before scaling to 5G complexity

This appendix documents the complete 4G testbed architecture, deployment procedures, and validation results.

The network stack consists of two dedicated machines running the backbone infrastructure: one running Open5GS for the EPC, while the other runs srsRAN 4G for the RAN. Readers can use this section as a guide for replicating the network, and as a foundation for understanding the experimental setup that enabled the 5G network development described in the main thesis.

This section will provide a comprehensive guide on how the 4G network was established, including a description of the equipment used for the setup and the software chosen for the network stack, the necessary setup steps and specific configurations applied to optimize performance, and the testing methodologies employed to validate the network's functionality and performance.

As an effort to make the network as reproducible as possible, the setup process will be detailed step-by-step, starting from the initial hardware assembly to the final testing

phase. This will include instructions on how to connect the various components, configure the network settings, and troubleshoot common issues that may arise during the setup process.

.1 System Architecture Overview

The 4G testbed is a self-contained LTE system comprising three main functional domains:

- **RAN**: Handles air-interface communication using srsRAN Evolved Node B (eNodeB) software running on a dedicated machine, along with a Nuand BladeRF SDR and appropriate antennas for signal transmission and reception
- **EPC**: Implements the core network functions using Open5GS software running on a separate machine, providing essential services such as authentication, user management, and data routing to the internet
- **UE**: Physical COTS device configured to connect to the RAN and access the services provided by the EPC

Figure .6 presents the logical architecture.

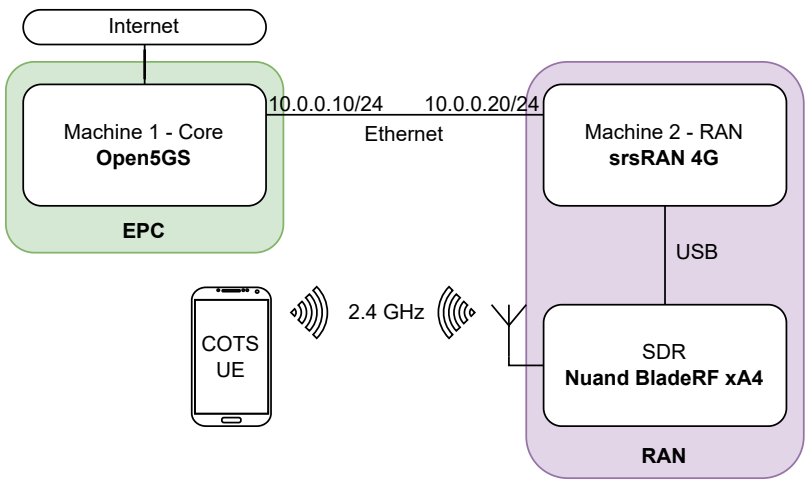


Figure .6: Logical architecture of the 4G network, illustrating the interactions between the RAN, EPC, and UE components.

.2 Hardware Components

The hardware used for the 4G network setup is composed of two identical machines, a Software-Defined Radio and antennas.

The computers running the 4G network are DELL Optiplex Micro 7020, with the following specifications:

Component	Machine 1	Machine 2
CPU	Intel Core i7-14700T	Intel Core i7-14700T
RAM	16 GB DDR5	16 GB DDR5
Operating System	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Storage	512 GB SSD	512 GB SSD

Table .1: Specifications of the machines used for the 4G network setup.

Data flows between the machines via a dedicated Ethernet backhaul network, composed of the two computers. The RAN on Machine 2 establishes a connection to the EPC on Machine 1 using Stream Control Transmission Protocol (SCTP).

The radio interface of the 4G network is implemented using a Nuand BladeRF xA4 SDR. This device connects the Machine 2 to provide the necessary radio capabilities for the RAN. In practical terms, the host machine executes the signal processing and network stack components, whereas the BladeRF xA4 manages the actual radio I/O. This separation of responsibilities improves flexibility, simplifies experimentation, and makes the overall testbed easier to implement andadapt to different deployment scenarios.

Attached to the xA4's SMA connectors are two LNAs, along with two omnidirectional antennas. These LNAs are used to amplify the TX/RX signals, improving the sensitivity of the RAN, expanding its coverage area and allowing for better performance in terms of signal quality.

Brand	Type	Model	Transmission
Nuand	LNA	BT-100	RX
Nuand	LNA	BT-200	TX

Table .2: Hardware components used for the radio interface in the 4G network setup.

Below, in Figure .7, shows the physical setup of the 4G network, including the two machines, the SDR, and the antennas. The photograph illustrates how the components are arranged in the lab environment, providing a visual reference for replicating the setup.

Figure .7: Photograph of the 4G network setup, showing the two machines, connected by an ethernet cable, the Nuand BladeRF xA4 SDR, and the antennas used for the radio interface.

.3 Software Components

The software stack of the network is composed mainly of three components:

- Open5GS, for the core network functions, running on Machine 1.
- srsRAN 4G, for the RAN, running on Machine 2.
- SoapySDR, bridging the connection between the BladeRF board and the srsRAN software, running on Machine 2.

.4 Connection between the machines

Before starting the setup, it is essential to first establish a connection between the two machines which will host the 4G network components. This connection is crucial as it enables the exchange of control and user data between the EPC and the RAN. The machines are connected via a ethernet cable, and the necessary network configurations are applied to ensure proper communication (.8).

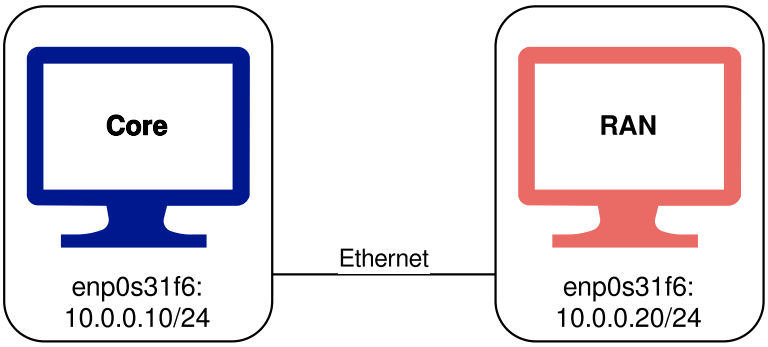


Figure .8: Diagram illustrating the connection between the two machines in the 4G network setup.

The steps to enable this connection are similar on both machines, differing only in the specific IP addresses assigned to each machine.

Connect both machines with an Ethernet cable and check the available network interfaces using the following command:

```
ip a
```

The output shows the available network interfaces, including the Ethernet interface (in this case enp0s31f6) (.9).

Figure .9: Output of the 'ip a' command, showing the available network interfaces, including the Ethernet interface (enp0s31f6).

Enable the Ethernet interface (if disabled) and assign a static IP address to it.

```
sudo ip link set enp0s31f6 up
sudo ip addr add 10.0.0.10/24 dev enp0s31f6
```

Core: 10.0.0.10/24 RAN: 10.0.0.20/24

To make the connection persistent across reboots, the network configuration needs to be added to the Netplan configuration file. This file may have a different name from the one in this example. Open the Netplan configuration file on `/etc/netplan/01-netcfg.yaml` using a text editor and add the following configuration to assign a static IP address to the Ethernet interface:

```
# Let NetworkManager manage all devices on this system
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    enp0s31f6:
      dhcp4: false
      addresses:
        - 10.0.0.10/24
```

After saving the changes, apply the new network configuration using the following command:

```
sudo netplan apply
```

Finally, verify that the machines can communicate with each other by pinging the assigned IP addresses. From Machine 1:

```
ping 10.0.0.20
```

The output should show successful ping responses, indicating that the machines are connected and can communicate with each other (.10).

Figure .10: Output of the ping command, showing successful responses between the two machines, indicating that they are connected and can communicate with each other.

Having established a successful connection between the two machines, the next step is to proceed with the installation and configuration of the software components necessary for the 4G network setup.

.5 Software Setup

.5.1 Open5GS

This section provides a step-by-step guide for installing and configuring Open5GS. The installation process comprises of the following steps:

1. Installing MongoDB.
2. Installing Open5GS.
3. Installing the WebUI for Open5GS.
4. Configuring Open5GS components (MME, HSS, SGW, PGW) according to the network requirements.

The necessary steps and more information about the project and installation steps can be found in the official Open5GS documentation [24]. Unless otherwise specified, all steps should be executed on the machine designated to run the EPC (Machine 1 in our setup).

.5.1.1 Installing MongoDB

Open5GS depends on MongoDB as its database backend, where it stores subscriber information, network configurations, and operational data. Therefore, the first step in setting up Open5GS is to install MongoDB:

```
sudo apt update
sudo apt install gnupg curl -y
curl -fsSL https://pgp.mongodb.com/server-8.0.asc | sudo gpg -o
    /usr/share/keyrings/mongodb-server-8.0.gpg --dearmor

echo "deb [ arch=amd64,arm64
    signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ]
    https://repo.mongodb.com/apt/ubuntu jammy/mongodb-org/8.0 multiverse" |
    sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list

sudo apt update
sudo apt install -y mongodb-org
```

After executing the above commands, MongoDB will be installed on the machine. We can then start and verify the installation of the MongoDB daemon:

```
sudo systemctl start mongod
sudo systemctl status mongod
```

In which the output should indicate that the MongoDB service is active and running:

```
core@core:~$ sudo systemctl status mongod
● mongod.service - MongoDB Database Server
     Loaded: loaded (/lib/systemd/system/mongod.service; enabled; vendor preset: enabled)
     Active: active (running) since Tue 2026-04-21 17:41:12 WEST; 1 week 3 days ago
       Docs: https://docs.mongodb.org/manual
```

Figure .11: Output of the command to check MongoDB service status, showing that the service is active and running.

To ensure MongoDB starts automatically on system boot, we can enable the service with the following command:

```
sudo systemctl enable mongod
```

For more detailed instructions and troubleshooting guidance, refer to the official MongoDB installation documentation [23].

.5.1.2 Installing Open5GS

After installing MongoDB, the next step is to add the Open5GS repository to the system and install Open5GS. This can be done by executing the following commands:

```
sudo add-apt-repository ppa:open5gs/latest
sudo apt update
sudo apt install open5gs -y
```

This will install the Open5GS package. After the installation is complete, we can verify that the Open5GS services are running correctly by checking their status:

```
sudo systemctl status open5gs-*
```

.5.1.3 Installing WebUI for Open5GS

Open5GS provides a WebUI for easier management of subscriber information, network configurations, and monitoring. The interface depends on Node.js, which needs to be installed first (on most Ubuntu builds, Node.js comes already pre-installed). To install Node.js, we can use the following commands:

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key |
sudo gpg --dearmor -o /etc/apt/keyrings/nodesource.gpg

NODE_MAJOR=20
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg]
    https://deb.nodesource.com/node_$NODE_MAJOR.x nodistro main" |
sudo tee /etc/apt/sources.list.d/nodesource.list

sudo apt update
sudo apt install nodejs -y
```

After installing Node.js, we can proceed to install the Open5GS WebUI by running the following command:

```
curl -fsSL https://open5gs.org/open5gs/assets/webui/install |
sudo -E bash -
```

The WebUI can be accessed by navigating to 'http://localhost:9999' in a web browser. The default login credentials are:

- Username: admin
- Password: 1423

.5.1.4 Configuring Open5GS Components

After installing Open5GS and the WebUI, the next step is to configure the core network components (MME, HSS, SGW, PGW) according to the network requirements and adhering to the legal regulations. In Open5GS, the configuration files for these components are located in the `/etc/open5gs/` directory, with each component having its own file.

First modify the MME configuration file, on `/etc/open5gs/mme.yaml`, to set the PLMN ID, TAC and IP address to attach the MME. Unless issued one by the national state regulator, use the international test PLMN ID (001/01), which is the one used in this configuration.

```
--- a/configs/open5gs/mme.yaml.in
+++ b/configs/open5gs/mme.yaml.in
@@ -10,7 +10,7 @@ mme:
     freeDiameter: @sysconfdir@/freeDiameter/mme.conf
     s1ap:
       server:
-        - address: 127.0.0.2
+        - address: 10.0.0.10
     gtpc:
       server:
         - address: 127.0.0.2
@@ -25,14 +25,14 @@ mme:
           port: 9090
     gummei:
       plmn_id:
-        mcc: 999
-        mnc: 70
+        mcc: 001
+        mnc: 01
       mme_gid: 2
       mme_code: 1
     tai:
       plmn_id:
-        mcc: 999
-        mnc: 70
+        mcc: 001
+        mnc: 01
```

```
      tac: 1
    security:
        integrity_order : [ EIA2, EIA1, EIA0 ]
```

Modify the SGWU configuration file, on `/etc/open5gs/sgwu.yaml`, to set the GTP-U IP address.

```
--- a/configs/open5gs/sgwu.yaml.in
+++ b/configs/open5gs/sgwu.yaml.in
@@ -15,7 +15,7 @@ sgwu:
 #          - address: 127.0.0.3
     gtpu:
       server:
-        - address: 127.0.0.6
+        - address: 10.0.0.10
```

After modifying the configuration files, restart the Open5GS daemons to apply all changes:

```
sudo systemctl restart open5gs-mmed
sudo systemctl restart open5gs-sgwud
```

Having completed the configuration of the Core components, the next step is to proceed with the installation and configuration of the RAN components, which will be covered in the next section.

.5.2 SDR Configuration

Before proceeding with the installation of srsRAN 4G, it is necessary to first configure the Nuand BladeRF xA4 SDR and install the necessary drivers to bridge the connection between the software and the hardware. This procedure consists of the following steps:

1. Setting up the Nuand drivers and firmware for the BladeRF xA4, with header files.
2. Installing SoapySDR.

.5.2.1 Setting up the Nuand drivers and firmware for the BladeRF xA4

Nuand provides official Ubuntu package repositories (PPAs) that simplify the installation of the BladeRF software in Ubuntu systems. Through these repositories, users can quickly install both the required host tools and libraries and the necessary firmware for the FPGA using `apt`, without needing to build everything manually from source. To enable the PPA:

```
sudo add-apt-repository ppa:nuandllc/bladerf
sudo apt update
```

After enabling the PPA, we can install the BladeRF software and firmware using the following command:

```
sudo apt install bladerf
sudo apt install bladerf-fpga-hostedx40
```

To bridge the connection between the BladeRF xA4 and the RAN, SoapySDR will be used, which will be covered in the next section (.5.2.2). For this to work, the header files for the BladeRF drivers need to be installed:

```
sudo apt install libbladerf-dev
```

With the BladeRF drivers installed, we can proceed into the installation of SoapySDR, which will allow us to use the BladeRF xA4 as the radio interface for the RAN.

.5.2.2 Installing SoapySDR

The SoapySDR component is comprised of two parts: the SoapySDR library, which is the main interface for all types of SDR devices, and the SoapyBladeRF plug-in, which provides specific support for the BladeRF xA4.

First, we need to install the necessary dependencies:

```
sudo apt install -y cmake g++ libpython3-dev python3-numpy swig
```

Afterwards, we can clone the SoapySDR repository and build the library:

```
git clone https://github.com/pothosware/SoapySDR.git
cd SoapySDR

mkdir build
cd build
cmake ..
make -j`nproc`
sudo make install -j`nproc`
sudo ldconfig
```

To verify the installation of the SoapySDR library, we can run the following command:

```
SoapySDRUtil --info
```

After installing SoapySDR, we can proceed to install the SoapyBladeRF plug-in, through their repository:

```
git clone https://github.com/pothosware/SoapyBladeRF.git
cd SoapyBladeRF

mkdir build
cd build
cmake ..
make
sudo make install
```

With both the SoapySDR library and the SoapyBladeRF plug-in installed, the BladeRF xA4 should now be recognized as an available SDR device, when running the following

command:

```
SoapySDRUtil --probe="driver=bladerf"
```

This command should output the details of the board, thus confirming that the BladeRF xA4 is properly configured and ready to be used as the radio interface for the RAN in our 4G network setup. With the SDR configuration complete, we can now proceed to install and configure srsRAN 4G.

.5.3 srsRAN 4G

First, install the dependencies for srsRAN 4G:

```
sudo apt-get install build-essential cmake libfftw3-dev libmbedtls-dev
    libboost-program-options-dev libconfig++-dev libsctp-dev
```

Then, clone the srsRAN repository and build the software:

```
git clone https://github.com/srsRAN/srsRAN_4G.git
cd srsRAN_4G
mkdir build
cd build
cmake ../
make
make test
```

Proceed to install srsRAN 4G:

```
sudo make install
srsran_install_configs.sh user
```

After installing srsRAN 4G, we need to modify the configuration files to set the correct IP address for the EPC and the correct parameters for the radio interface. The configuration files are located in the `/usr/local/etc/srsran/` directory, with separate files for the EPC and the eNodeB. With everything ready, we can proceed to run the network, which will be covered in the next section (.6).

.6 Running the network

.6.1 Preparing the system

To ensure the best performance on the RAN side, the srsRAN team provices a performance script to optimize the system for real-time processing. This script should be executed before running the RAN components, on Machine 2:

```
# Download and execute the srsRAN performance tuning script
curl -fsSL
    https://raw.githubusercontent.com/srsran/srsRAN_Project/main/scripts/srsran_performance
    | bash
```

On Machine 1, we need to add a route to the UE subnet, so that the EPC can route the traffic from the UE to the internet:

```
### Enable IPv4/IPv6 Forwarding
$ sudo sysctl -w net.ipv4.ip_forward=1
$ sudo sysctl -w net.ipv6.conf.all.forwarding=1

### Add NAT Rule
$ sudo iptables -t nat -A POSTROUTING -s 10.45.0.0/16 ! -o ogstun -j
    MASQUERADE
$ sudo ip6tables -t nat -A POSTROUTING -s 2001:db8:cafe::/48 ! -o ogstun -j
    MASQUERADE
```

Now, to start the network, we need to first start the EPC on Machine 2. On one command prompt window, run the following command:

```
sudo srsepc
```

Machine 2 should now connect to Machine 1, and srsRAN should recognize the Open5GS network functions running on the other Machine. In another command prompt window, start the eNodeB on Machine 2:

```
sudo srsenb
```

Now the RAN is up and running, and the UE can connect to the network.

.7 Troubleshooting

Provisioning the eSIM and connecting to the network

In this chapter, we will provision an eSIM profile for the testbench and connect to the network. We will create an eSIM profile through a provisioning server and follow the steps to activate it on our testbench. Once the eSIM is activated, we will verify the connection to the network and ensure that we can send and receive data. This process is crucial for testing the performance of our system under real network conditions.

.8 Creating an eSIM profile

To create an eSIM profile for our testbench, we will use Simlessly, specifically their Remote SIM Provisioning (RSP) tool, which allows custom eSIM profile creation. This platform requires an account to access the provisioning services. Once logged in, the user is greeted with a dashboard:

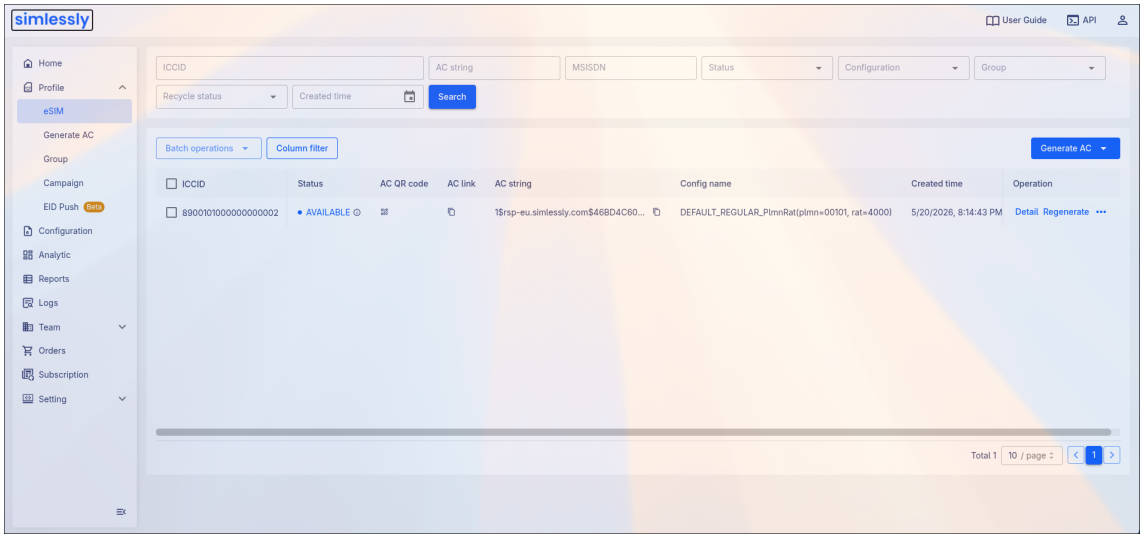


Figure .12: Simlessly Dashboard

From the dashboard, we can click the "Generate AC" button to create a new eSIM profile.

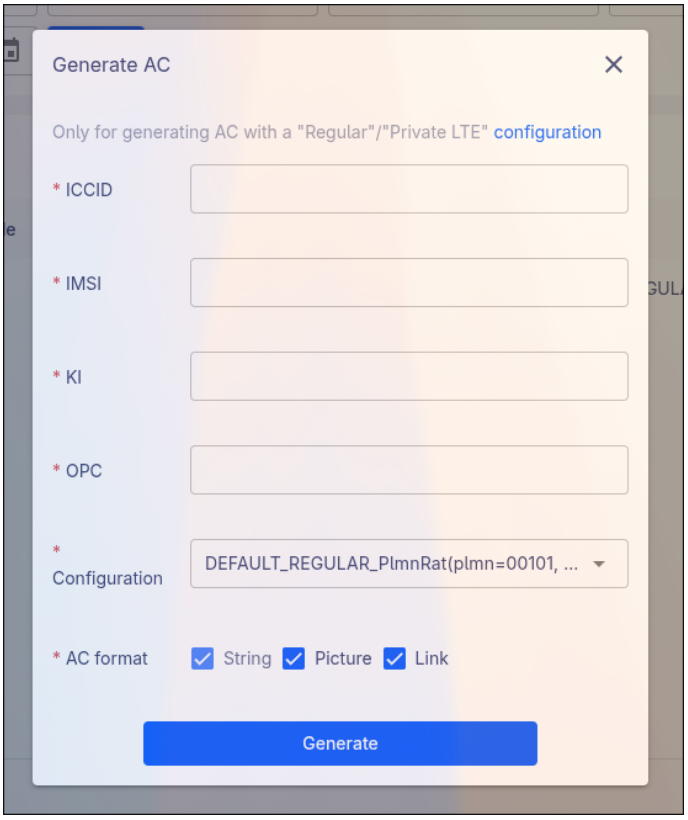


Figure .13: Generating an eSIM profile

After clicking the button, we are prompted to fill in the details for the eSIM profile.

For the testbench, we will use the following keys:

Key	Value
ICCID	8900101000000000002
IMSI	001010000000002
KI	465B5CE8B199B49FAA5F0A2EE238A6BC
OPC	E8ED289DEBA952E4B4C389C94229606A

Table .3: eSIM profile details

Click on "Generate" to create the eSIM profile. Once generated, back on the dashboard, click on the QR code icon on the "AC QR code" column to display the Activation Code:

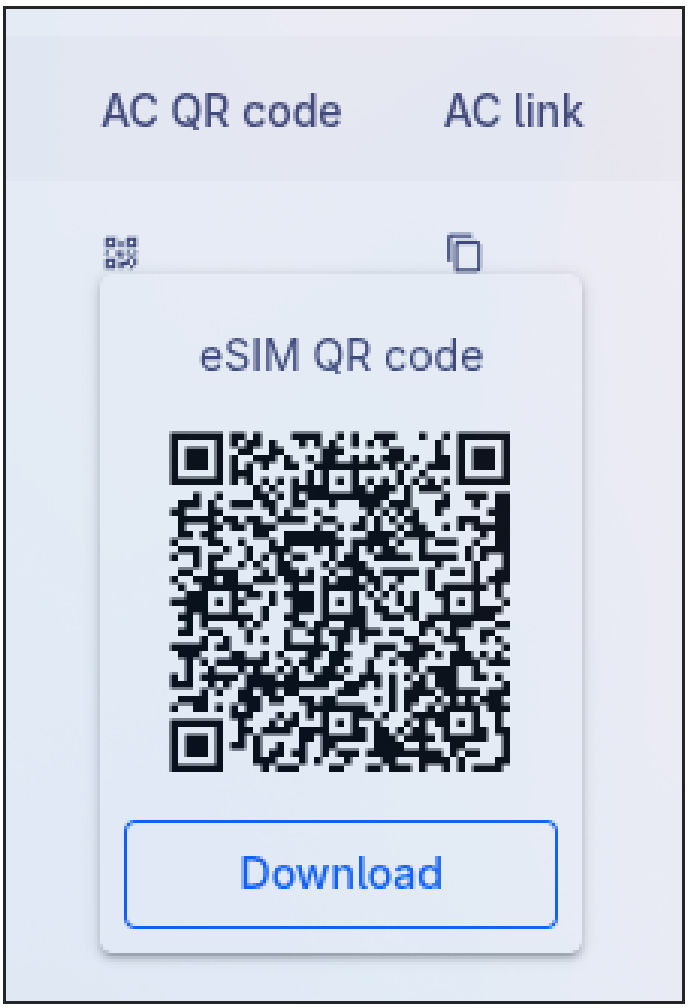


Figure .14: eSIM Activation Code